

TÀI LIỆU ÔN TẬP LPIC-1 (Exam 101-500) — 120 CÂU HỎI

Nguồn: Bộ 120 câu hỏi trắc nghiệm LPIC 101-500 (Topic 1). Câu hỏi và đáp án được giữ nguyên bản tiếng Anh (đúng như đề thi thật trên ExamTopics), kèm theo bản dịch nghĩa tiếng Việt ngay bên dưới, và phần giải thích chi tiết (lý do chọn đáp án đúng, lý do loại đáp án sai, từ khóa ghi nhớ) bằng tiếng Việt.

Cách dùng: Đọc câu hỏi (tiếng Anh) → tự trả lời trước → đối chiếu bản dịch để chắc chắn hiểu đúng đề → xem đáp án và phần giải thích → học từ khóa ghi nhớ để nhớ lâu.

Chú thích ký hiệu:

- ✓ = Đáp án đúng
- ✗ = Đáp án sai (kèm lý do loại)
- 🔑 = Từ khóa ghi nhớ

Câu 1. Loại filesystem mặc định của `mkfs`

Question: Which type of file system is created by `mkfs` when it is executed with the block device name only and without any additional parameters? (Dịch câu hỏi: Khi chạy `mkfs` chỉ với tên block device (không có tham số nào khác) thì nó tạo ra loại filesystem nào?)

- A. XFS
- B. VFAT
- C. ext2
- D. ext3
- E. ext4

Dịch nghĩa các đáp án:

- A. XFS
- B. VFAT
- C. ext2
- D. ext3
- E. ext4

✓ **Answer: C — ext2**

Giải thích: `mkfs` không có tham số `-t` sẽ gọi mặc định `mkfs.ext2`. Đây là hành vi mặc định lâu đời của công cụ `mkfs` trên hầu hết distro — nếu muốn tạo loại khác phải chỉ định rõ, ví dụ `mkfs -t ext4` hoặc dùng thẳng `mkfs.ext4`, `mkfs.xfs` ...

🔑 **Từ khóa:** "mkfs tron = ext2" (không có `-t` → mặc định cổ điển nhất: ext2).

Câu 2. umask cho thư mục mới

Question: Which umask value ensures that new directories can be read, written and listed by their owning user, read and listed by their owning group and are not accessible at all for everyone else? (Dịch câu hỏi: Giá trị umask nào đảm bảo thư mục mới: chủ sở hữu đọc/ghi/liệt kê (rwx), nhóm đọc/liệt kê (r-x), người khác không có quyền gì?)

- A. 0750
- B. 0027
- C. 0036
- D. 7640
- E. 0029

Dịch nghĩa các đáp án:

- A. 0750
- B. 0027
- C. 0036
- D. 7640
- E. 0029

✅ **Answer: B — 0027**

Giải thích: Thư mục mặc định có quyền 0777 (rwxrwxrwx). umask là phần quyền bị **trừ đi**. Ta muốn kết quả cuối là `rwxr-x---` (750). Vậy $\text{umask} = 777 - 750 = 027$ (phép trừ theo từng bit octal, không phải trừ số học thường, nhưng ở đây khớp cả hai cách).

- Owner: 7 (không trừ gì) $\rightarrow$ umask bit owner = 0
- Group: cần r-x (5), gốc là 7 $\rightarrow$ trừ 2 (mất quyền ghi w)
- Other: cần --- (0), gốc là 7 $\rightarrow$ trừ 7

$\rightarrow$ umask = 027.

🔑 **Từ khóa:** `umask = 777 (dir mặc định) - quyền mong muốn`. Nhớ: thư mục gốc luôn là 777, file là 666.

Câu 3. `tune2fs` — số ngày trước khi fsck

Question: Which of the following commands changes the number of days before the ext3 filesystem on /dev/sda1 has to run through a full filesystem check while booting? (Dịch câu hỏi: Lệnh nào thay đổi số ngày trước khi filesystem ext3 trên `/dev/sda1` phải chạy full fsck khi boot?)

- A. `tune2fs -d 200 /dev/sda1`
- B. `tune2fs -i 200 /dev/sda1`
- C. `tune2fs -c 200 /dev/sda1`

- D. `tune2fs -n 200 /dev/sda1`
- E. `tune2fs --days 200 /dev/sda1`

Dịch nghĩa các đáp án:

- A. `tune2fs -d 200 /dev/sda1`
- B. `tune2fs -i 200 /dev/sda1`
- C. `tune2fs -c 200 /dev/sda1`
- D. `tune2fs -n 200 /dev/sda1`
- E. `tune2fs --days 200 /dev/sda1`

✅ **Answer: B — `tune2fs -i 200 /dev/sda1`**

Giải thích: `tune2fs -i` (interval) đặt **khoảng thời gian tối đa** (theo ngày, hoặc thêm hậu tố d/m/w) giữa hai lần fsck bắt buộc. Còn `-c` (count) là đặt theo **số lần mount** tối đa trước khi fsck, không phải theo thời gian.

❌ C (`-c 200`) — đây là giới hạn theo **số lần mount**, không phải số ngày → dễ nhầm với B.

🔑 **Từ khóa:** `-i` = interval = thời gian (ngày); `-c` = count = số lần mount.

Câu 4. Dung lượng dự trữ mặc định cho root trên ext4

Question: Which is the default percentage of reserved space for the root user on new ext4 filesystems?
(Dịch câu hỏi: Tỷ lệ phần trăm mặc định dung lượng dự trữ cho user root trên filesystem ext4 mới là bao nhiêu?)

- A. 10%
- B. 3%
- C. 15%
- D. 0%
- E. 5%

Dịch nghĩa các đáp án:

- A. 10%
- B. 3%
- C. 15%
- D. 0%
- E. 5%

✅ **Answer: E — 5%**

Giải thích: Khi tạo ext2/ext3/ext4 bằng `mkfs`, mặc định 5% dung lượng được dự trữ (reserved blocks) chỉ dành cho root, giúp hệ thống không bị "full disk" hoàn toàn (root vẫn ghi log, sửa lỗi được). Có thể chỉnh bằng `tune2fs -m <percent>`.

🔑 **Từ khóa:** "ext-mặc-định-5%" — `tune2fs -m` chỉnh **margin/reserved** %.

Câu 5. Mount thủ công filesystem không có trong /etc/fstab (systemd)

Question: Which of the following is true when a file system, which is neither listed in /etc/fstab nor known to system, is mounted manually? (Dịch câu hỏi: Điều gì đúng khi một filesystem không nằm trong `/etc/fstab` và hệ thống chưa biết, được mount thủ công?)

- A. systemd ignores any manual mounts which are not done using the `systemctl mount` command
- B. The command `systemctl mountsync` can be used to create a mount unit based on the existing mount
- C. systemd automatically generates a mount unit and monitors the mount point without changing it
- D. Unless a systemd mount unit is created, systemd unmounts the file system after a short period of time
- E. `systemctl unmount` must be used to remove the mount because system opens a file descriptor on the mount point

Dịch nghĩa các đáp án:

- A. systemd bỏ qua các mount thủ công không dùng `systemctl mount`
- B. Có lệnh `systemctl mountsync` tạo mount unit từ mount hiện có
- C. systemd tự động sinh ra một mount unit và theo dõi mount point mà không thay đổi nó
- D. Trừ khi tạo mount unit, systemd sẽ unmount fs sau một thời gian ngắn
- E. Phải dùng `systemctl unmount` để gỡ mount

✅ **Answer: C — systemd automatically generates a mount unit and monitors the mount point without changing it**

Giải thích: systemd liên tục quét `/proc/self/mountinfo`; khi phát hiện mount mới (kể cả mount thủ công bằng `mount`), nó tự tạo một **transient mount unit** tương ứng để theo dõi trạng thái, nhưng không can thiệp/thay đổi mount đó.

❌ D — sai vì systemd **không** tự ý unmount những gì bạn mount tay.

🔑 **Từ khóa:** systemd = "theo dõi thụ động", tự sinh unit ảo (transient), không tự unmount.

Câu 6. [FILL BLANK] Lệnh cập nhật database cho `locate`

Question: FILL BLANK - Which program updates the database that is used by the `locate` command? (Specify ONLY the command without any path or parameters). (Dịch câu hỏi: Chương trình nào cập nhật database được `locate` sử dụng? (chỉ ghi tên lệnh))

✅ **Answer:** `updatedb`

Giải thích: `locate` tìm file cực nhanh vì nó tra cứu trong một **database** (thường ở `/var/lib/mlocate/mlocate.db`) thay vì quét ổ đĩa trực tiếp như `find`. Database này được sinh/cập nhật định kỳ (qua cron) bằng lệnh `updatedb`.

🔑 **Từ khóa:** `locate` đọc database ↔ `updatedb` ghi database. Cron thường chạy `updatedb` mỗi ngày.

Câu 7. `mount --bind` dùng để làm gì

Question: What does the command `mount --bind` do? (Dịch câu hỏi: Lệnh `mount --bind` làm gì?)

- A. It makes the contents of one directory available in another directory
- B. It mounts all available filesystems to the current directory
- C. It mounts all user mountable filesystems to the user's home directory
- D. It mounts all file systems listed in `/etc/fstab` which have the option `userbind` set
- E. It permanently mounts a regular file to a directory

Dịch nghĩa các đáp án:

- A. Làm cho nội dung của một thư mục xuất hiện ở một thư mục khác
- B. Mount tất cả filesystem khả dụng vào thư mục hiện tại
- C. Mount tất cả filesystem user có thể mount vào home directory
- D. Mount các fs trong `/etc/fstab` có option `userbind`
- E. Mount vĩnh viễn 1 file thường vào 1 thư mục

✅ **Answer: A — It makes the contents of one directory available in another directory**

Giải thích: `mount --bind /nguồn /dịch` gắ n (map) nội dung thư mục nguồ n vào một điểm khác trong cây thư mục — hai đường dẫn cùng trỏ tới cùng dữ liệu thật (giống hard link nhưng ở cá p thư mục và cá p mount, hoạt động cả qua ranh giới filesystem/chroot).

🔑 **Từ khóa:** `bind mount` = "chiế u" một thư mục ra vị trí khác, không copy dữ liệu.

Câu 8. Tạo file mới có cùng inode với `a.txt`

Question: Consider the following output from the command `ls -li`: How would a new file named `c.txt` be created with the same inode number as `a.txt` (Inode 525385)? (Dịch câu hỏi: Làm sao tạo file `c.txt` có cùng số inode với `a.txt` (inode 525385)?)

- A. `ln -h a.txt c.txt`
- B. `ln c.txt a.txt`
- C. `ln a.txt c.txt`
- D. `ln -f c.txt a.txt`
- E. `ln -i 525385 c.txt`

Dịch nghĩa các đáp án:

- A. `ln -h a.txt c.txt`
- B. `ln c.txt a.txt`
- C. `ln a.txt c.txt`
- D. `ln -f c.txt a.txt`
- E. `ln -i 525385 c.txt`

✅ **Answer: C — `ln a.txt c.txt`**

Giải thích: `ln <target> <linkname>` tạo **hard link** — thứ tự tham số là: **nguồn trước, tên link mới sau**. Hard link trở cùng inode, nên `ls -i` sẽ cho ra cùng số inode. Không có tùy chọn `-i` để "gán" inode number tùy ý như đáp án E gợi ý (bịa).

🔑 **Từ khóa:** `ln nguồn đích_mới` — giới ng cú pháp `cp nguồn đích`.

Câu 9. Đảm bảo file mới trong thư mục thuộc group `sales`

Question: Consider the following directory: `drwxrwxr-x 2 root sales 4096 Jan 1 15:21 sales` Which command ensures new files created within the directory sales are owned by the group sales? (Choose two.) (Dịch câu hỏi: Thư mục `sales` có quyền `drwxrwxr-x root:sales`. Lệnh nào đảm bảo file mới tạo trong đó thuộc group `sales`? (Chọn 2))

- A. `chmod g+s sales`
- B. `setpol -R newgroup=sales sales`
- C. `chgrp -p sales sales`
- D. `chown --persistent *.sales sales`
- E. `chmod 2775 sales`

Dịch nghĩa các đáp án:

- A. `chmod g+s sales`
- B. `setpol -R newgroup=sales sales`
- C. `chgrp -p sales sales`
- D. `chown --persistent *.sales sales`
- E. `chmod 2775 sales`

✅ **Answer: A, E**

Giải thích: Đây chính là **SGID (SetGID) trên thư mục** — khi bật, mọi file/thư mục con tạo mới bên trong sẽ tự động kế thừa group của thư mục cha thay vì group hiệu lực của người tạo.

- A: `chmod g+s` = bật bit SGID (ký hiệu).
- E: `chmod 2775` = bit "2" ở đầu (octal) chính là SGID, cộng với quyền 775.
- B, C, D là lệnh/option bịa, không tồn tại.

🔑 **Từ khóa:** SGID trên **thư mục** → file con "thừa kế" group cha. `2xxx` hoặc `g+s`.

Câu 10. Hiển thị tất cả filesystem đang mount

Question: In order to display all currently mounted filesystems, which of the following commands could be used? (Choose two.) (Dịch câu hỏi: Lệnh nào hiển thị tất cả filesystem đang được mount? (Chọn 2))

- A. `cat /proc/self/mounts`
- B. `free`
- C. `lsmounts`
- D. `mount`
- E. `cat /proc/filesystems`

Dịch nghĩa các đáp án:

- A. `cat /proc/self/mounts`
- B. `free`
- C. `lsmounts`
- D. `mount`
- E. `cat /proc/filesystems`

✅ **Answer: A, D**

Giải thích: `mount` (không tham số) liệt kê toàn bộ mount hiện tại; `/proc/self/mounts` (hoặc `/proc/mounts`) là file ảo chứa đúng thông tin đó ở dạng máy đọc được.

❌ E — `/proc/filesystems` chỉ liệt kê **loại** filesystem mà kernel hỗ trợ (ext4, xfs, vfat...), không phải các mount hiện tại. ❌ B `free` là RAM/swap. ❌ C lệnh không tồn tại.

🔑 **Từ khóa:** `/proc/filesystems` = loại fs kernel biết; `/proc/mounts` = mount đang chạy.

Câu 11. [FILL BLANK] Lệnh xem dung lượng đĩa đang dùng

Question: FILL BLANK - Which command displays the current disk space usage for all mounted file systems? (Specify ONLY the command without any path or parameters.) (Dịch câu hỏi: Lệnh nào hiển thị dung lượng đĩa đang sử dụng của tất cả filesystem đã mount?)

✅ **Answer:** `df`

Giải thích: `df` (disk free/disk filesystem) báo cáo dung lượng theo **filesystem** (tổng, đã dùng, còn trống, % dùng). Phân biệt với `du` (disk usage) là dung lượng theo **file/thư mục**.

🔑 **Từ khóa:** `df` = disk filesystem (theo ổ đĩa); `du` = disk usage (theo file/thư mục).

Câu 12. **chown** đổi cả owner và group

Question: Which chown command changes the ownership to dave and the group to staff on a file named data.txt? (Dịch câu hỏi: Lệnh **chown** nào đổi owner thành **dave** và group thành **staff** cho file **data.txt**?)

- A. `chown dave/staff data.txt`
- B. `chown -u dave -g staff data.txt`
- C. `chown --user dave --group staff data.txt`
- D. `chown dave+staff data.txt`
- E. `chown dave:staff data.txt`

Dịch nghĩa các đáp án:

- A. `chown dave/staff data.txt`
- B. `chown -u dave -g staff data.txt`
- C. `chown --user dave --group staff data.txt`
- D. `chown dave+staff data.txt`
- E. `chown dave:staff data.txt`

✅ **Answer: E — `chown dave:staff data.txt`**

Giải thích: Cú pháp chuẩn: `chown [owner][:group] file`. Dấu hai chấm `:` ngăn cách owner và group (có thể dùng dấu chấm `.` ở hệ thống cũ, nhưng `:` là chuẩn POSIX hiện đại).

🔑 **Từ khóa:** `chown user:group file` — dấu **hai chấm**, không phải `/`, `+`, hay option `-u -g`.

Câu 13. Lý do không nên dùng hard link

Question: When considering the use of hard links, what are valid reasons not to use hard links? (Dịch câu hỏi: Lý do hợp lệ để KHÔNG dùng hard link là gì?)

- A. Hard links are not available on all Linux systems because traditional filesystems, such as ext4, do not support them
- B. Each hard link has individual ownership, permissions and ACLs which can lead to unintended disclosure of file content
- C. Hard links are specific to one filesystem and cannot point to files on another filesystem
- D. If users other than root should be able to create hard links, suLN has to be installed and configured
- E. When a hard linked file is changed, a copy of the file is created and consumes additional space

Dịch nghĩa các đáp án:

- A. Hard link không có trên mọi hệ thống vì ext4 không hỗ trợ (sai — ext4 hỗ trợ)
- B. Mỗi hard link có owner/permission/ACL riêng (sai — chúng dùng chung 1 inode nên chung permission)
- C. Hard link chỉ giới hạn trong 1 filesystem, không thể trỏ sang file ở filesystem khác

- **D.** Cần cài `sudo` để user thường tạo hard link (bịa)
- **E.** Khi sửa file hard link, một bản sao được tạo ra (sai — đó là copy-on-write, không phải hard link)

✅ **Answer: C — Hard links are specific to one filesystem and cannot point to files on another filesystem**

Giải thích: Hard link chỉ là một tên khác trỏ tới **cùng inode**. Vì số inode chỉ có ý nghĩa (duy nhất) trong phạm vi một filesystem/partition, hard link **không thể** vượt qua ranh giới filesystem (khác với symbolic link — có thể trỏ tới bất kỳ đâu, kể cả khác ổ đĩa).

🔑 **Từ khóa:** Hard link = giới hạn **trong 1 filesystem**; Symlink = **không giới hạn**, trỏ đi đâu cũng được (kể cả sai đường dẫn).

Câu 14. Vị trí man pages theo FHS

Question: In compliance with the FHS, in which of the directories are man pages found? (*Dịch câu hỏi: Theo FHS, man page nằm ở thư mục nào?*)

- **A.** `/opt/man/`
- **B.** `/usr/doc/`
- **C.** `/usr/share/man/`
- **D.** `/var/pkg/man`
- **E.** `/var/man/`

Dịch nghĩa các đáp án:

- **A.** `/opt/man/`
- **B.** `/usr/doc/`
- **C.** `/usr/share/man/`
- **D.** `/var/pkg/man`
- **E.** `/var/man/`

✅ **Answer: C — `/usr/share/man/`**

Giải thích: FHS (Filesystem Hierarchy Standard) quy định dữ liệu **kiến trúc-độc-lập** (architecture-independent, chỉ đọc) như tài liệu, man page, locale... nằm dưới `/usr/share/`. Do đó man page ở `/usr/share/man/`.

🔑 **Từ khóa:** `/usr/share/` = dữ liệu dùng chung, không phụ thuộc kiến trúc CPU (docs, man, locale, icons...).

Câu 15. [FILL BLANK] File trong /proc chứa tham số kernel boot

Question: FILL BLANK - Which file in the /proc filesystem lists parameters passed from the bootloader to the kernel? (Specify the file name only without any path.) (*Dịch câu hỏi: File nào trong /proc liệt kê các tham số được bootloader truyền cho kernel?*)

✅ **Answer:** `cmdline`

Giải thích: `/proc/cmdline` chứa dòng lệnh kernel đầy đủ mà GRUB (hoặc bootloader khác) đã truyền vào lúc boot, ví dụ `root=/dev/sda1 ro quiet splash`. Rất hữu ích khi debug lỗi boot hoặc kiểm tra tham số kernel đang chạy.

🔑 **Từ khóa:** `cat /proc/cmdline` → xem chính xác GRUB đã truyền gì cho kernel.

Câu 16. PID của tiến trình `init` (System V)

Question: What is the process ID number of the init process on a System V init based system? (*Dịch câu hỏi: PID của tiến trình `init` trên hệ System V init là bao nhiêu?*)

- A. -1
- B. 0
- C. 1
- D. It is different with each reboot
- E. It is set to the current run level

Dịch nghĩa các đáp án:

- A. -1
- B. 0
- C. 1
- D. Khác nhau mỗi lần reboot
- E. Bằng runlevel hiện tại

✅ **Answer:** C — 1

Giải thích: `init` (hoặc `systemd` trên hệ hiện đại) luôn là tiến trình **đầu tiên** được kernel khởi chạy sau khi boot xong, nên PID cố định là **1**. Mọi tiến trình khác đều là con/cháu của PID 1.

🔑 **Từ khóa:** PID 1 = `init/systemd` = "tổ tiên" của mọi process.

Câu 17. Daemon quản lý sự kiện nguồn điện

Question: Which daemon handles power management events on a Linux system? (Dịch câu hỏi: Daemon nào xử lý sự kiện quản lý nguồn điện (power management)?)

- A. acpid
- B. batteryd
- C. pwrmgntd
- D. psd
- E. inetd

Dịch nghĩa các đáp án:

- A. `acpid`
- B. `batteryd`
- C. `pwrmgntd`
- D. `psd`
- E. `inetd`

✅ **Answer: A — acpid**

Giải thích: `acpid` lắng nghe sự kiện ACPI (Advanced Configuration and Power Interface) từ kernel — như nhấn nút nguồn, đóng nắp laptop, cắm/rút sạc — rồi kích hoạt script tương ứng. Các đáp án khác đều là tên bịa (trừ `inetd` là daemon mạng, không liên quan nguồn điện).

🔑 **Từ khóa:** ACPI events → **acpid**.

Câu 18. Boot sequence với BIOS

Question: Which of the following statements are true about the boot sequence of a PC using a BIOS? (Choose two.) (Dịch câu hỏi: Phát biểu nào đúng về quá trình boot PC dùng BIOS? (Chọn 2))

- A. Some parts of the boot process can be configured from the BIOS
- B. Linux does not require the assistance of the BIOS to boot a computer
- C. The BIOS boot process starts only if secondary storage, such as the hard disk, is functional
- D. The BIOS initiates the boot process after turning the computer on
- E. The BIOS is started by loading hardware drivers from secondary storage, such as the hard disk

Dịch nghĩa các đáp án:

- A. Một phần của boot process có thể cấu hình từ BIOS
- B. Linux không cần BIOS hỗ trợ để boot
- C. BIOS chỉ chạy nếu ổ cứng hoạt động tốt
- D. BIOS khởi động quá trình boot ngay sau khi bật máy
- E. BIOS được nạp bằng driver phần cứng từ ổ cứng

✅ **Answer: A, D**

Giải thích: BIOS nằm trên chip firmware của mainboard, tự chạy **ngay khi bật nguồn** (POST — Power-On Self-Test), độc lập với việc ổ cứng còn hoạt động hay không. Nhiệm vụ thiết lập boot (thứ tự ổ đĩa boot, bật/tắt thiết bị...) được cấu hình ngay trong BIOS setup.

❌ B sai — Linux **cần** BIOS/UEFI để tìm và nạp bootloader ban đầu. ❌ C, E sai logic — BIOS chạy **trước khi** đọc gì từ ổ cứng, không phụ thuộc ổ cứng.

🔑 **Từ khóa:** BIOS = firmware chạy đầu tiên, độc lập ổ cứng; có thể chỉnh trong BIOS setup.

Câu 19. Đặc điểm firmware UEFI

Question: What is true regarding UEFI firmware? (Choose two.) (*Dịch câu hỏi: Điều gì đúng về firmware UEFI? (Chọn 2)*)

- **A.** It can read and interpret partition tables
- **B.** It can use and read certain file systems
- **C.** It stores its entire configuration on the /boot/ partition
- **D.** It is stored in a special area within the GPT metadata
- **E.** It is loaded from a fixed boot disk position

Dịch nghĩa các đáp án:

- **A.** Có thể đọc và hiểu partition table
- **B.** Có thể dùng/đọc được một số filesystem
- **C.** Lưu toàn bộ cấu hình trên partition /boot
- **D.** Lưu trong vùng đặc biệt của GPT metadata
- **E.** Nạp từ vị trí cố định trên ổ đĩa boot

✅ **Answer: A, B**

Giải thích: UEFI hiện đại hơn BIOS: nó **hiểu được GPT/MBR partition table** và có thể **đọc trực tiếp filesystem FAT32** trên ESP (EFI System Partition) để tìm và chạy file `.efi` — đây là lý do UEFI hỗ trợ boot menu đồ họa, boot trực tiếp từ file thay vì đọc sector thô như BIOS.

❌ C, D — cấu hình UEFI (NVRAM) nằm trong chip firmware của mainboard, không nằm trên GPT hay `/boot/`. ❌ E — UEFI không phụ thuộc "vị trí cố định" kiểu MBR (đó là đặc trưng BIOS/MBR).

🔑 **Từ khóa:** UEFI = "thông minh hơn" — đọc được partition table + filesystem (FAT), khác BIOS chỉ đọc sector thô.

Câu 20. Ngăn module lỗi tự load khi boot

Question: A faulty kernel module is causing issues with a network interface card. Which of the following actions ensures that this module is not loaded automatically when the system boots? (*Dịch câu hỏi: Module kernel lỗi gây vấn đề cho card mạng. Hành động nào đảm bảo module đó không tự load khi boot?*)

- **A.** Using `lsmod --remove --autoclean` without specifying the name of a specific module
- **B.** Using `modinfo -k` followed by the name of the offending module
- **C.** Using `modprobe -r` followed by the name of the offending module
- **D.** Adding a blacklist line including the name of the offending module to the file `/etc/modprobe.d/blacklist.conf`
- **E.** Deleting the kernel module's directory from the file system and recompiling the kernel, including its modules

Dịch nghĩa các đáp án:

- **A.** `lsmod --remove --autoclean`
- **B.** `modinfo -k <module>`
- **C.** `modprobe -r <module>`
- **D.** Thêm dòng blacklist vào `/etc/modprobe.d/blacklist.conf`
- **E.** Xóa thư mục module và biên dịch lại kernel

✅ **Answer: D — Adding a blacklist line including the name of the offending module to the file `/etc/modprobe.d/blacklist.conf`**

Giải thích: File cấu hình trong `/etc/modprobe.d/` cho phép **blacklist** một module — ngăn nó được tự động load bởi udev/kernel module autoloading, dù module vẫn còn tồn tại trên đĩa (vẫn có thể load tay bằng `modprobe` nếu cần).

- ❌ **C** `modprobe -r` chỉ gỡ module đang chạy ở phiên hiện tại — **không** ngăn nó load lại ở lần boot sau.
- ❌ **E** quá cực đoan, không thực tế / không cần thiết.

🔑 **Từ khóa:** Ngăn load vĩnh viễn = **blacklist** trong `/etc/modprobe.d/`. `modprobe -r` chỉ là gỡ tạm thời.

Câu 21. Khi nào kernel ring buffer bị reset

Question: When is the content of the kernel ring buffer reset? (Choose two.) (*Dịch câu hỏi: Nội dung kernel ring buffer bị reset khi nào? (Chọn 2)*)

- **A.** When the ring buffer is explicitly reset using the command `dmesg --clear`
- **B.** When the ring buffer is read using `dmesg` without any additional parameters
- **C.** When a configurable amount of time, 15 minutes by default, has passed
- **D.** When the kernel loads a previously unloaded kernel module
- **E.** When the system is shut down or rebooted

Dịch nghĩa các đáp án:

- **A.** Khi reset thủ công bằng `dmesg --clear`
- **B.** Khi đọc bằng `dmesg` không tham số
- **C.** Sau khoảng thời gian cấu hình (mặc định 15 phút)
- **D.** Khi kernel nạp 1 module chưa từng load
- **E.** Khi hệ thống shutdown hoặc reboot

✅ **Answer: A, E**

Giải thích: Ring buffer là vùng nhớ (RAM) tạm để kernel ghi log — nên nó **mất khi tắt máy/khởi động lại** (E). Nó cũng có thể xóa thủ công bằng `dmesg --clear` hoặc `dmesg -c` (clear sau khi đọc) (A).

❌ B — chỉ **đọc** (`dmesg` suông) không xóa gì cả. ❌ C, D — không có cơ chế tự động timeout hay reset khi load module.

🔑 **Từ khóa:** Ring buffer sống trong RAM → mất khi reboot; xóa tay bằng `dmesg --clear/-c`.

Câu 22. Chương trình đầu tiên kernel chạy (System V init)

Question: What is the first program the Linux kernel starts at boot time when using System V init?
(Dịch câu hỏi: Chương trình đầu tiên kernel Linux khởi chạy lúc boot khi dùng System V init là gì?)

- **A.** `/lib/init.so`
- **B.** `/proc/sys/kernel/init`
- **C.** `/etc/rc.d/rcinit`
- **D.** `/sbin/init`
- **E.** `/boot/init`

Dịch nghĩa các đáp án:

- **A.** `/lib/init.so`
- **B.** `/proc/sys/kernel/init`
- **C.** `/etc/rc.d/rcinit`
- **D.** `/sbin/init`
- **E.** `/boot/init`

✅ **Answer: D — /sbin/init**

Giải thích: Sau khi kernel khởi tạo phân cứng và mount root filesystem, nó tìm và chạy chương trình `init` — theo thứ tự thử các đường dẫn mặc định, phổ biến nhất là `/sbin/init`. Đây chính là tiến trình PID 1 (xem lại Câu 16).

🔑 **Từ khóa:** Kernel → `/sbin/init` → PID 1.

Câu 23. Tìm gói sở hữu một file (Debian)

Question: A Debian package creates several files during its installation. Which of the following commands searches for packages owning the file `/etc/debian_version`? (*Dịch câu hỏi: Lệnh nào tìm gói Debian nào sở hữu file `/etc/debian_version`?*)

- A. `apt-get search /etc/debian_version`
- B. `apt -r /etc/debian_version`
- C. `find /etc/debian_version -dpkg`
- D. `dpkg -S /etc/debian_version`
- E. `apt-file /etc/debian_version`

Dịch nghĩa các đáp án:

- A. `apt-get search /etc/debian_version`
- B. `apt -r /etc/debian_version`
- C. `find /etc/debian_version -dpkg`
- D. `dpkg -S /etc/debian_version`
- E. `apt-file /etc/debian_version`

✅ **Answer: D — `dpkg -S /etc/debian_version`**

Giải thích: `dpkg -S <file>` (Search) tra cứu **ngược**: cho biết gói nào đã cài đặt sở hữu một file cụ thể trên hệ thống. Đây là công cụ tra cứu cục bộ trên gói **đã cài**, khác với `apt-file` (cần cài thêm, tra cứu cả gói **chưa** cài từ repository — và cú pháp đúng phải là `apt-file search`, không phải như E viết).

🔑 **Từ khóa:** `dpkg -S file` = Search package chứa file này (đã cài rồi).

Câu 24. Nội dung của EFI System Partition

Question: What is contained on the EFI System Partition? (*Dịch câu hỏi: EFI System Partition (ESP) chứa gì?*)

- A. The Linux root file system
- B. The first stage boot loader
- C. The default swap space file
- D. The Linux default shell binaries
- E. The user home directories

Dịch nghĩa các đáp án:

- A. Root filesystem của Linux
- B. First stage bootloader
- C. File swap mặc định
- D. Binary shell mặc định
- E. Home directory người dùng

✅ **Answer: B — The first stage boot loader**

Giải thích: ESP là một partition FAT32 nhỏ chứa các file `.efi` (bootloader giai đoạn đầu, ví dụ `grubx64.efi`, `bootmgfw.efi` ...) mà UEFI firmware đọc trực tiếp để khởi động hệ điều hành.

🔑 **Từ khóa:** ESP = FAT32 nhỏ chứa file `.efi` (bootloader), không chứa root fs hay dữ liệu người dùng.

Câu 25. Thư mục chứa shared library (64-bit)

Question: Which of the following directories on a 64 bit Linux system typically contain shared libraries? (Choose two.) (Dịch câu hỏi: Thư mục nào trên hệ 64-bit thường chứa shared library? (Chọn 2))

- A. `~/lib64/`
- B. `/usr/lib64/`
- C. `/var/lib64/`
- D. `/lib64/`
- E. `/opt/lib64/`

Dịch nghĩa các đáp án:

- A. `~/lib64/`
- B. `/usr/lib64/`
- C. `/var/lib64/`
- D. `/lib64/`
- E. `/opt/lib64/`

✅ **Answer: B, D**

Giải thích: Theo FHS, thư viện dùng chung thiết yếu cho hệ thống nằm ở `/lib64/` (cần có sẵn khi boot, trước khi mount `/usr`), còn thư viện cho ứng dụng người dùng cài thêm nằm ở `/usr/lib64/`. Hậu tố `64` phân biệt với thư viện 32-bit tương ứng ở `/lib/`, `/usr/lib/`.

🔑 **Từ khóa:** `/lib64` (hệ thống lõi) + `/usr/lib64` (ứng dụng) — cặp đôi kinh điển như `/bin` và `/usr/bin`.

Câu 26. File tồn tại trong cài đặt GRUB 2 chuẩn

Question: Which of the following files exist in a standard GRUB 2 installation? (Choose two.) (Dịch câu hỏi: File nào tồn tại trong một bản cài GRUB 2 chuẩn? (Chọn 2))

- A. `/boot/grub/stages/stage0`
- B. `/boot/grub/i386-pc/lvm.mod`
- C. `/boot/grub/fstab`
- D. `/boot/grub/grub.cfg`
- E. `/boot/grub/linux/vmlinuz`

Dịch nghĩa các đáp án:

- A. `/boot/grub/stages/stage0`
- B. `/boot/grub/i386-pc/lvm.mod`
- C. `/boot/grub/fstab`
- D. `/boot/grub/grub.cfg`
- E. `/boot/grub/linux/vmlinuz`

✅ **Answer: B, D**

Giải thích: GRUB 2 lưu file cấu hình chính `grub.cfg` (do `grub-mkconfig` / `update-grub` sinh ra) và các **module** (`.mod`) theo từng nền tảng, ví dụ thư mục `i386-pc/` chứa module cho hệ BIOS x86, trong đó có `lvm.mod` (hỗ trợ đọc LVM).

❌ A `stage0` / `fstab` / `vmlinuz` trong `/boot/grub/` là khái niệm của **GRUB Legacy (0.9x)** hoặc không tồn tại — GRUB 2 dùng cấu trúc module thay vì "stage1/stage1.5/stage2".

🔑 **Từ khóa:** GRUB 2 = `grub.cfg` + thư mục module (`*.mod`) theo platform, KHÔNG còn "stage0/1/2" như GRUB Legacy.

Câu 27. Cài tất cả gói tên kết thúc bằng "foo" (zypper)

Question: Which of the following commands installs all packages with a name ending with the string foo? (Dịch câu hỏi: Lệnh nào cài tất cả gói có tên kết thúc bằng chuỗi "foo"?)

- A. `zypper get '*foo'`
- B. `zypper update 'foo?'`
- C. `zypper force 'foo*'`
- D. `zypper install '*foo'`
- E. `zypper add '*.foo'`

Dịch nghĩa các đáp án:

- A. `zypper get '*foo'`
- B. `zypper update 'foo?'`
- C. `zypper force 'foo*'`
- D. `zypper install '*foo'`
- E. `zypper add '*.foo'`

✅ **Answer: D — zypper install '*foo'**

Giải thích: `zypper install` hỗ trợ wildcard shell-style (`*`, `?`) trong tên gói. `*foo` nghĩa là "bất kỳ chuỗi nào + kết thúc bằng foo". Các subcommand `get`, `force`, `add` không tồn tại trong zypper.

🔑 **Từ khóa:** zypper dùng wildcard `*` / `?` kiểu shell glob, không phải regex.

Câu 28. Thuộc tính cần đổi khi clone VM

Question: Which of the following properties of a Linux system should be changed when a virtual machine is cloned? (Choose two.) (*Dịch câu hỏi: Thuộc tính nào nên đổi khi clone một máy ảo Linux? (Chọn 2)*)

- A. The partitioning scheme
- B. The file system
- C. The D-Bus Machine ID
- D. The permissions of /root/
- E. The SSH host keys

Dịch nghĩa các đáp án:

- A. Sơ đồ phân vùng
- B. Filesystem
- C. D-Bus Machine ID
- D. Quyền của `/root/`
- E. SSH host keys

✅ **Answer: C, E**

Giải thích: Khi clone VM, hai thứ **định danh duy nhất** dễ gây xung đột nếu giữ nguyên là:

- **D-Bus Machine ID** (`/etc/machine-id`) — dùng để định danh máy trong hệ thống D-Bus/systemd, hai máy trùng ID gây lỗi.
- **SSH host keys** — nếu 2 máy dùng chung key, dễ bị cảnh báo MITM hoặc mất tính xác thực duy nhất.

Phân vùng, filesystem, quyền `/root/` là cấu hình hệ thống, không cần đổi khi clone.

🔑 **Từ khóa:** Clone VM → đổi **machine-id** + **SSH host keys** (2 thứ định danh máy).

Câu 29. Cài GRUB 2 vào MBR của ổ cứng thứ ba

Question: Which of the following commands installs GRUB 2 into the master boot record on the third hard disk? (*Dịch câu hỏi: Lệnh nào cài GRUB 2 vào master boot record của ổ cứng thứ ba?*)

- A. grub2 install /dev/sdc
- B. grub-mkrescue /dev/sdc
- C. grub-mbrinstall /dev/sdc
- D. grub-setup /dev/sdc
- E. grub-install /dev/sdc

Dịch nghĩa các đáp án:

- A. `grub2 install /dev/sdc`

- B. `grub-mkrescue /dev/sdc`
- C. `grub-mbrinstall /dev/sdc`
- D. `grub-setup /dev/sdc`
- E. `grub-install /dev/sdc`

✅ **Answer: E — grub-install /dev/sdc**

Giải thích: `grub-install <thiết bị>` cài GRUB vào MBR (hoặc ESP với UEFI) của **cả ổ đĩa** — chú ý dùng tên thiết bị (`/dev/sdc`), không phải tên partition (`/dev/sdc1`). Ổ thứ 3 trong quy ước đặt tên Linux (sda, sdb, sdc...) là `sdc`.

🔑 **Từ khóa:** `grub-install /dev/sdX` (ổ đĩa, không phải partition) → cài vào MBR.

Câu 30. Loại partition dùng cho Linux swap

Question: Which of the following partition types is used for Linux swap spaces when partitioning hard disk drives? (*Dịch câu hỏi: Loại partition (partition type ID) nào dùng cho Linux swap khi phân vùng ổ đĩa?*)

- A. 7
- B. 82
- C. 83
- D. 8e
- E. fd

Dịch nghĩa các đáp án:

- A. 7
- B. 82
- C. 83
- D. 8e
- E. fd

✅ **Answer: B — 82**

Giải thích: Trong bảng partition type ID (hệ MBR, dùng bởi `fdisk`): `82` = Linux swap, `83` = Linux (filesystem thường), `8e` = Linux LVM, `fd` = Linux RAID autodetect, `7` = NTFS/exFAT (Windows).

🔑 **Từ khóa:** 82 = swap, 83 = Linux fs thường, 8e = LVM, fd = RAID — cần thuộc lòng 4 mã này.

Câu 31. Cấu hình yum

Question: What is true regarding the configuration of yum? (Choose two.) (*Dịch câu hỏi: Điều gì đúng về cấu hình yum? (Chọn 2)*)

- **A.** Changes to the repository configuration become active after running yum confupdate
- **B.** Changes to the yum configuration become active after restarting the yumd service
- **C.** The configuration of package repositories can be divided into multiple files
- **D.** Repository configurations can include variables such as \$basearch or \$releasever
- **E.** In case /etc/yum.repos.d/ contains files, /etc/yum.conf is ignored

Dịch nghĩa các đáp án:

- **A.** Cấu hình repo có hiệu lực sau khi chạy `yum confupdate` (bịa, không cần lệnh này)
- **B.** Cấu hình yum có hiệu lực sau khi restart service `yumd` (yum không phải daemon thường trực)
- **C.** Cấu hình repository package có thể chia thành nhiều file
- **D.** Cấu hình repo có thể dùng biến như `$basearch` hoặc `$releasever`
- **E.** Nếu `/etc/yum.repos.d/` có file thì `/etc/yum.conf` bị bỏ qua (sai)

✅ **Answer: C, D**

Giải thích: yum đọc `/etc/yum.conf` (cấu hình chung) và mọi file `.repo` trong `/etc/yum.repos.d/` (mỗi repo một file riêng, hoặc gộp) — cả hai cùng được dùng, không loại trừ nhau. Trong file `.repo`, có thể dùng biến định sẵn như `$releasever` (phiên bản OS) và `$basearch` (kiến trúc CPU) để URL tự thích ứng.

🔑 **Từ khóa:** yum = `/etc/yum.conf` (chung) + `/etc/yum.repos.d/*.repo` (từng repo), hỗ trợ biến `$basearch`, `$releasever`. Không cần restart service nào — yum không phải daemon.

Câu 32. apt-get subcommand nâng cấp toàn bộ gói lên bản mới nhất

Question: Which of the following apt-get subcommands installs the newest versions of all currently installed packages? (*Dịch câu hỏi: Subcommand `apt-get` nào cài phiên bản mới nhất cho tất cả gói đã cài?*)

- **A.** auto-update
- **B.** dist-upgrade
- **C.** full-upgrade
- **D.** install
- **E.** update

Dịch nghĩa các đáp án:

- **A.** `auto-update`
- **B.** `dist-upgrade`
- **C.** `full-upgrade`
- **D.** `install`
- **E.** `update`

✅ **Answer: B — dist-upgrade**

Giải thích: `apt-get update` chỉ làm mới danh sách gói (metadata), không cài gì. `apt-get upgrade` nâng cấp gói nhưng **không** cài/gỡ gói mới để giải quyết phụ thuộc. `apt-get dist-upgrade` nâng cấp **toàn bộ**, kể cả khi cần cài thêm/gỡ bớt gói để giải quyết dependency (tương đương `full-upgrade` ở `apt`) — nhưng câu hỏi hỏi cụ thể `apt-get`, nên đáp án đúng là `dist-upgrade`).

 **Từ khóa:** `apt-get update` = làm mới index; `upgrade` = nâng cấp an toàn; `dist-upgrade` = nâng cấp toàn diện (đổi cả dependency).

Câu 33. Gỡ gói nhưng giữ lại file cấu hình (dpkg)

Question: Which command uninstalls a package but keeps its configuration files in case the package is re-installed? (Dịch câu hỏi: Lệnh nào gỡ một gói nhưng giữ lại file cấu hình để phòng khi cài lại?)

- A. `dpkg -s pkgname`
- B. `dpkg -L pkgname`
- C. `dpkg -P pkgname`
- D. `dpkg -v pkgname`
- E. `dpkg -r pkgname`

Dịch nghĩa các đáp án:

- A. `dpkg -s`
- B. `dpkg -L`
- C. `dpkg -P`
- D. `dpkg -v`
- E. `dpkg -r`

 **Answer: E — `dpkg -r pkgname`**

Giải thích: `dpkg -r` (remove) chỉ gỡ **binary/chương trình**, giữ lại file cấu hình (`/etc/...`) — hữu ích nếu định cài lại sau và muốn giữ config cũ. Ngược lại `dpkg -P` (purge) gỡ **luôn cả file cấu hình**.

 **Từ khóa:** `-r` = remove (giữ config); `-P` = purge (xóa sạch, kể cả config).

Câu 34. Liệt kê dependency của file RPM

Question: Which of the following commands lists the dependencies of the RPM package file `foo.rpm`? (Dịch câu hỏi: Lệnh nào liệt kê các dependency của file gói `foo.rpm`?)

- A. `rpm -qpR foo.rpm`
- B. `rpm -dep foo`
- C. `rpm -ld foo.rpm`
- D. `rpm -R foo.rpm`
- E. `rpm -pD foo`

Dịch nghĩa các đáp án:

- A. `rpm -qpR foo.rpm`
- B. `rpm -dep foo`
- C. `rpm -ld foo.rpm`
- D. `rpm -R foo.rpm`
- E. `rpm -pD foo`

✅ **Answer: A — rpm -qpR foo.rpm**

Giải thích: Ghép các option: `-q` (query), `-p` (chỉ định file .rpm cụ thể chưa cài), `-R` (Requires — liệt kê dependency). Đây là tổ hợp chuẩn để xem một file RPM **chưa cài đặt** cần những gì để cài được.

🔑 **Từ khóa:** `rpm -q` = hỏi, `-p` = từ file package, `-R` = requires (deps). Nhớ combo `-qpR`.

Câu 35. Giá trị niceness tối đa user thường được gán

Question: What is the maximum niceness value that a regular user can assign to a process with the nice command when executing a new process? (Dịch câu hỏi: Giá trị niceness tối đa mà user thường (không phải root) có thể gán cho tiến trình bằng `nice`?)

- A. 9
- B. 15
- C. 19
- D. 49
- E. 99

Dịch nghĩa các đáp án:

- A. 9
- B. 15
- C. 19
- D. 49
- E. 99

✅ **Answer: C — 19**

Giải thích: Thang độ ưu tiên `nice` chạy từ **-20 (ưu tiên cao nhất)** đến **19 (ưu tiên thấp nhất)**. User thường chỉ được phép làm tiến trình "nhường nhịn hơn" — tức tăng giá trị nice (0 → 19), **không** được giảm xuống dưới 0 (chỉ root mới có quyền tăng độ ưu tiên, tức đặt số âm).

🔑 **Từ khóa:** nice: -20 (ưu tiên nhất) ... 0 (mặc định) ... 19 (thấp nhất). User thường chỉ tăng (0→19), root mới giảm được (âm).

Câu 36. Liệt kê file/thư mục trong /tmp thuộc user root

Question: Which of the following commands list all files and directories within the /tmp/ directory and its subdirectories which are owned by the user root? (Choose two.) (Dịch câu hỏi: *Lệnh nào liệt kê tất cả file/thư mục trong /tmp/ (và thư mục con) thuộc sở hữu user root? (Chọn 2)*)

- A. `find /tmp -user root -print`
- B. `find -path /tmp -uid root`
- C. `find /tmp -uid root -print`
- D. `find /tmp -user root`
- E. `find -path /tmp -user root -print`

Dịch nghĩa các đáp án:

- A. `find /tmp -user root -print`
- B. `find -path /tmp -uid root`
- C. `find /tmp -uid root -print`
- D. `find /tmp -user root`
- E. `find -path /tmp -user root -print`

✅ **Answer: A, D**

Giải thích: Cú pháp đúng của `find` là `find <đường_dẫn> <biểu_thức>`, và option lọc theo **tên** user là `-user` (VD `-user root`). Option `-uid` yêu cầu **số** **UID**, không nhận tên "root" trực tiếp theo cú pháp chuẩn POSIX test → B, C sai vì dùng `-uid root` (chuỗi tên) thay vì số. `-print` là hành động mặc định nên có/không có `-print` không ảnh hưởng kết quả hiển thị → A và D tương đương nhau, đều đúng.

❌ B, E dùng `-path /tmp` (sai vị trí cú pháp, không phải cách chỉ định thư mục gốc tìm kiếm).

🔑 **Từ khóa:** `find /tmp -user root` — luôn ghi đường dẫn ngay sau `find`; `-user` nhận **tên**, `-uid` nhận **số**.

Câu 37. Toán tử redirect hợp lệ trong Bash

Question: Which of the following are valid stream redirection operators within Bash? (Choose two.) (Dịch câu hỏi: *Toán tử redirect stream nào hợp lệ trong Bash? (Chọn 2)*)

- A. `<`
- B. `#>`
- C. `%>`
- D. `>>>`
- E. `2>&1`

Dịch nghĩa các đáp án:

- A. `<`

- B. `#>`
- C. `%>`
- D. `>>>`
- E. `2>&1`

✅ **Answer: A, E**

Giải thích: `<` là redirect input chuẩn (stdin từ file). `2>&1` là redirect **file descriptor 2 (stderr)** sang cùng đích với **file descriptor 1 (stdout)** — cú pháp rất hay gặp trong `command > log 2>&1`. Các toán tử `#>`, `%>`, `>>>` không tồn tại trong Bash.

🔑 **Từ khóa:** `2>&1` = "gộp stderr vào stdout" — luôn đặt SAU redirect stdout (`> file 2>&1`).

Câu 38. Lệnh vi xóa 2 dòng (dòng hiện tại + dòng kế)

Question: Which of the following vi commands deletes two lines, the current and the following line? (Dịch câu hỏi: Lệnh vi nào xóa 2 dòng: dòng hiện tại và dòng kế tiếp?)

- A. `d2`
- B. `2d`
- C. `2dd`
- D. `dd2`
- E. `de12`

Dịch nghĩa các đáp án:

- A. `d2`
- B. `2d`
- C. `2dd`
- D. `dd2`
- E. `de12`

✅ **Answer: C — 2dd**

Giải thích: Trong vi/vim, cú pháp lập lệnh là **số lần**. `dd` xóa 1 dòng, nên `2dd` = xóa 2 dòng (dòng hiện tại + dòng kế). Số luôn đặt **trước** lệnh, không phải sau.

🔑 **Từ khóa:** vi = `[số] [lệnh]`, ví dụ `2dd` (xóa 2 dòng), `3yy` (copy 3 dòng), `5j` (xuống 5 dòng).

Câu 39. Chạy nền và không bị kill khi logout

Question: The command `dbmaint &` was used to run `dbmaint` in the background. However, `dbmaint` is terminated after logging out of the system. Which alternative `dbmaint` invocation lets `dbmaint` continue to run even when the user running the program logs out? (Dịch câu hỏi: `dbmaint &` chạy nền nhưng bị kill khi logout. Cách nào giúp nó tiếp tục chạy kể cả khi logout?)

- A. job -b dmaint
- B. dbmaint &>/dev/pts/null
- C. nohup dbmaint &
- D. bg dbmaint
- E. wait dbmaint

Dịch nghĩa các đáp án:

- A. `job -b dmaint`
- B. `dbmaint &>/dev/pts/null`
- C. `nohup dbmaint &`
- D. `bg dbmaint`
- E. `wait dbmaint`

✅ **Answer: C — nohup dbmaint &**

Giải thích: Khi logout, shell gửi tín hiệu **SIGHUP** (hangup) tới các tiến trình con, thường làm chúng bị kill. `nohup` bắt (ignore) tín hiệu SIGHUP cho tiến trình chỉ định, giúp nó "sống sót" sau khi phiên đăng nhập kết thúc. Kết hợp `&` để chạy nền ngay từ đầu.

🔑 **Từ khóa:** Logout → SIGHUP → `nohup` = "miễn nhiễm" với SIGHUP.

Câu 40. Chạy script trực tiếp không tạo subshell

Question: From a Bash shell, which of the following commands directly execute the instructions from the file `/usr/local/bin/runme.sh` without starting a subshell? (Choose two.) (Dịch câu hỏi: Lệnh nào chạy trực tiếp các lệnh trong file `/usr/local/bin/runme.sh` mà KHÔNG tạo subshell? (Chọn 2))

- A. `source /usr/local/bin/runme.sh`
- B. `/usr/local/bin/runme.sh`
- C. `/bin/bash /usr/local/bin/runme.sh`
- D. `./usr/local/bin/runme.sh`
- E. `run /usr/local/bin/runme.sh`

Dịch nghĩa các đáp án:

- A. `source /usr/local/bin/runme.sh`
- B. `/usr/local/bin/runme.sh`
- C. `/bin/bash /usr/local/bin/runme.sh`
- D. `./usr/local/bin/runme.sh`
- E. `run /usr/local/bin/runme.sh`

✅ **Answer: A, D**

Giải thích: `source file` và dấu chấm `. file` là hai cách viết tương đương — cả hai đều thực thi script ngay trong shell hiện tại (không fork subshell mới), nên mọi biến/thay đổi thư mục trong script sẽ ảnh hưởng tới shell gọi nó.

✗ B, C — chạy script bằng đường dẫn trực tiếp hoặc gọi `bash script.sh` để tạo **process con (subshell/child process)** mới để thực thi.

🔑 **Từ khóa:** `source file` $\equiv$ `. file` = chạy ngay trong shell hiện tại (không subshell). Còn `./file` hoặc `bash file` = subshell mới.

Câu 41. [FILL BLANK] Chạy lệnh lặp lại theo khoảng thời gian

Question: FILL BLANK - Which program runs a command in specific intervals and refreshes the display of the program's output? (Specify ONLY the command without any path or parameters.) (Dịch câu hỏi: Chương trình nào chạy 1 lệnh theo khoảng thời gian định kỳ và làm mới màn hình hiển thị kết quả?)

✅ **Answer:** `watch`

Giải thích: `watch -n <giây> <lệnh>` chạy lại lệnh liên tục (mặc định mỗi 2 giây), xóa và vẽ lại màn hình mỗi lần — rất hữu ích để theo dõi output thay đổi theo thời gian thực, ví dụ `watch df -h`.

🔑 **Từ khóa:** `watch <lệnh>` = "theo dõi" output cập nhật liên tục trên màn hình.

Câu 42. Ký tự vi để dán (paste) nội dung vừa xóa xuống dưới dòng hiện tại

Question: Immediately after deleting 3 lines of text in vi and moving the cursor to a different line, which single character command will insert the deleted content below the current line? (Dịch câu hỏi: Ngay sau khi xóa 3 dòng trong vi và di chuyển con trỏ tới dòng khác, ký tự đơn nào chèn nội dung đã xóa **xuống dưới** dòng hiện tại?)

- A. i (lowercase)
- B. p (lowercase)
- C. P (uppercase)
- D. U (uppercase)
- E. u (lowercase)

Dịch nghĩa các đáp án:

- A. `i`
- B. `p`
- C. `P`
- D. `U`
- E. `u`

✅ **Answer:** B — p (lowercase)

Giải thích: Trong vi, **p** (paste, chữ thường) dán nội dung buffer **sau/dưới** dòng hoặc ký tự hiện tại; **P** (chữ hoa) dán **trước/trên**. **u** là undo, **U** là undo toàn bộ thay đổi trên dòng, **i** là chuyển sang chế độ insert (không liên quan dán).

Từ khóa: **p** = paste sau (dưới); **P** = Paste trước (trên). Chữ thường/hoa đứng ở vị trí.

Câu 43. Chuyển CR-LF sang LF bằng **tr**

Question: Which of the following commands changes all CR-LF line breaks in the text file userlist.txt to Linux standard LF line breaks and stores the result in newlist.txt? (Dịch câu hỏi: Lệnh nào đổi toàn bộ line break CR-LF (Windows) trong `userlist.txt` sang LF (Linux), lưu vào `newlist.txt`?)

- A. `tr -d '\r' < userlist.txt > newlist.txt`
- B. `tr -c '\n\r' " userlist.txt`
- C. `tr '\r\n' " newlist.txt`
- D. `tr '\r' '\n' userlist.txt newlist.txt`
- E. `tr -s '/M/J/' userlist.txt newlist.txt`

Dịch nghĩa các đáp án:

- A. `tr -d '\r' < userlist.txt > newlist.txt`
- B. `tr -c '\n\r' 'userlist.txt`
- C. `tr '\r\n' 'newlist.txt`
- D. `tr '\r' '\n' userlist.txt newlist.txt`
- E. `tr -s '/^M/^J/' userlist.txt userlist.txt`

✓ **Answer: A — `tr -d '\r' < userlist.txt > newlist.txt`**

Giải thích: CR-LF = `\r\n`. Muốn thành LF (`\n`) thì chỉ cần **xóa hẵn ký tự `\r`** (vì `\n` vẫn giữ nguyên) — dùng `tr -d '\r'` (delete). `tr` chỉ đọc từ stdin/ghi ra stdout nên phải dùng redirect `<` và `>`, không nhận tên file làm tham số trực tiếp (loại D vì cú pháp sai).

Từ khóa: CRLF → LF: chỉ cần `tr -d '\r'` (xóa CR, giữ LF). `tr` luôn cần `< in > out`, không nhận filename trực tiếp.

Câu 44. Regex sed thêm hậu tố ".bak" trước ".txt"

Question: Given the following input stream: `txt1.txt atxt.txt txtB.txt` Which of the following regular expressions turns this input stream into the following output stream? `txt1.bak.txt atxt.bak.txt txtB.bak.txt` (Dịch câu hỏi: Input: `txt1.txt atxt.txt txtB.txt`. Regex nào biến nó thành `txt1.bak.txt atxt.bak.txt txtB.bak.txt`?)

- A. `s/^.txt/.bak/`
- B. `s/txt/bak.txt/`

- C. `s/txt$/bak.txt/`
- D. `s/txt$/.bak/`
- E. `s/[.txt]/.bak$1/`

Dịch nghĩa các đáp án:

- A. `s/^ .txt/ .bak/`
- B. `s/txt/bak.txt/`
- C. `s/txt$/bak.txt/`
- D. `s/^txt$/.bak^/`
- E. `s/[.txt]/.bak$1/`

✅ **Answer: C — `s/txt$/bak.txt/`**

Giải thích: Cần thay thế đúng chuỗi "txt" nằm ở **cuối** (phần đuôi mở rộng `.txt`) bằng "bak.txt" — dấu `$` neo (anchor) vào **cuối chuỗi**, đảm bảo chỉ thay "txt" cuối cùng (phần extension), không đụng tới "txt" xuất hiện ở đầu tên file (như trong "txtB" hay "atxt" nếu nó nằm giữa).

🔑 **Từ khóa:** `^` = neo đầu dòng, `$` = neo cuối dòng. Muốn sửa đúng phần đuôi file → luôn dùng `$`.

Câu 45. Lưu file hiện tại trong vi với tên khác trước khi thoát

Question: Which command must be entered before exiting vi to save the current file as filea.txt? (Dịch câu hỏi: Lệnh nào cần gõ trước khi thoát vi để lưu file hiện tại thành `filea.txt`?)

- A. `%s filea.txt`
- B. `%w filea.txt`
- C. `:save filea.txt`
- D. `:w filea.txt`
- E. `:s filea.txt`

Dịch nghĩa các đáp án:

- A. `%s filea.txt`
- B. `%w filea.txt`
- C. `:save filea.txt`
- D. `:w filea.txt`
- E. `:s filea.txt`

✅ **Answer: D — `:w filea.txt`**

Giải thích: Trong vi, chế độ lệnh (`:`) dùng `w` (write) để lưu, có thể kèm tên file để **"Save As"**: `:w tên_file` ghi nội dung hiện tại ra file mới mà không đổi tên file đang mở. `:s` là lệnh substitute (thay thế văn bản), không phải save.

 **Từ khóa:** `:w` = write (save); `:w tenfile` = Save As; `:s` = substitute (KHÔNG phải save — dễ nhầm).

Câu 46. Tín hiệu gửi khi nhấn Ctrl+C

Question: Which of the following signals is sent to a process when the key combination Ctrl+C is pressed on the keyboard? (Dịch câu hỏi: Tín hiệu nào được gửi tới tiến trình khi nhấn Ctrl+C?)

- A. SIGTERM
- B. SIGCONT
- C. SIGSTOP
- D. SIGKILL
- E. SIGINT

Dịch nghĩa các đáp án:

- A. SIGTERM
- B. SIGCONT
- C. SIGSTOP
- D. SIGKILL
- E. SIGINT

 **Answer: E — SIGINT**

Giải thích: Ctrl+C gửi **SIGINT** (Interrupt) — yêu cầu tiến trình dừng lại một cách "lịch sự" (có thể bị chương trình bắt và xử lý riêng). Khác với Ctrl+Z gửi **SIGTSTP/SIGSTOP** (tạm dừng, xem Câu 95).

 **Từ khóa:** Ctrl+C = SIGINT (ngắt t); Ctrl+Z = SIGTSTP (tạm dừng); Ctrl+\ = SIGQUIT.

Câu 47. Vừa hiện output ra màn hình vừa ghi ra file

Question: Which of the following commands displays the output of the foo command on the screen and also writes it to a file called /tmp/foodata? (Dịch câu hỏi: Lệnh nào hiển thị output của `foo` trên màn hình và ghi vào `/tmp/foodata`?)

- A. `foo | less /tmp/foodata`
- B. `foo | cp /tmp/foodata`
- C. `foo > /tmp/foodata`
- D. `foo | tee /tmp/foodata`
- E. `foo > stdout >> /tmp/foodata`

Dịch nghĩa các đáp án:

- A. `foo | less /tmp/foodata`

- B. `foo | cp /tmp/foodata`
- C. `foo > /tmp/foodata`
- D. `foo | tee /tmp/foodata`
- E. `foo > stdout >> /tmp/foodata`

✅ **Answer: D — `foo | tee /tmp/foodata`**

Giải thích: `tee` đọc từ stdin, vừa **in ra stdout** (để hiện trên màn hình) vừa **ghi ra file** chỉ định — như chữ T phân nhánh đường ống dữ liệu. Đáp án C chỉ redirect ra file (không hiện màn hình).

🔑 **Từ khóa:** `tee` = "chia đôi" luồng dữ liệu: vừa hiện màn hình, vừa lưu file.

Câu 48. `echo` với chuỗi trong nháy đơn có biến \$USER

Question: What output will be displayed when the user fred executes the following command? `echo 'fred $USER'` (Dịch câu hỏi: User `fred` chạy `echo 'fred $USER'`. Output là gì?)

- A. fred fred
- B. fred /home/fred/
- C. 'fred \$USER'
- D. fred \$USER
- E. 'fred fred'

Dịch nghĩa các đáp án:

- A. `fred fred`
- B. `fred /home/fred/`
- C. `'fred $USER'`
- D. `fred $USER`
- E. `'fred fred'`

✅ **Answer: D — `fred $USER`**

Giải thích: Nháy đơn (`'...'`) trong Bash **ngăn mọi mở rộng biến** (không như nháy kép `"..."` cho phép biến được thay giá trị). Vì vậy `$USER` được in ra **y nguyên như văn bản**, không được thay bằng `fred`.

🔑 **Từ khóa:** Nháy đơn `'...'` = literal (không mở rộng biến); nháy kép `"..."` = có mở rộng biến `$VAR`.

Câu 49. Hiển thị đường dẫn thực thi sẽ được chạy

Question: Which of the following commands displays the path to the executable file that would be executed when the command `foo` is invoked? (Dịch câu hỏi: Lệnh nào hiển thị đường dẫn tới file thực thi sẽ chạy khi gọi lệnh `foo`?)

- A. `lsattr foo`
- B. `apropos foo`
- C. `locate foo`
- D. `whatis foo`
- E. `which foo`

Dịch nghĩa các đáp án:

- A. `lsattr foo`
- B. `apropos foo`
- C. `locate foo`
- D. `whatis foo`
- E. `which foo`

✅ **Answer: E — `which foo`**

Giải thích: `which` tìm trong các thư mục liệt kê ở biến `$PATH` (theo thứ tự) và in ra đường dẫn đầy đủ của file thực thi đầu tiên khớp tên — chính là cái sẽ được chạy khi gõ lệnh đó. `whatis` / `apropos` tra cứu man page, `locate` tìm theo database toàn hệ thống (không quan tâm `$PATH`), `lsattr` xem thuộc tính file ext.

🔑 **Từ khóa:** `which` = tìm theo `$PATH` → "lệnh này thực chất chạy file nào".

Câu 50. Option của `find` khi kết hợp `xargs` với tên file có khoảng trắng

Question: When redirecting the output of `find` to the `xargs` command, what option to `find` is useful if the filenames contain spaces? (Dịch câu hỏi: Khi redirect output của `find` sang `xargs`, option nào của `find` hữu ích nếu tên file có khoảng trắng?)

- A. `-rep-space`
- B. `-printnul`
- C. `-nospace`
- D. `-ignore-space`
- E. `-print0`

Dịch nghĩa các đáp án:

- A. `-rep-space`
- B. `-printnul`
- C. `-nospace`
- D. `-ignore-space`
- E. `-print0`

✅ **Answer: E — `-print0`**

Giải thích: `find -print0` in tên file, ngăn cách bằng ký tự NUL (`\0`) thay vì newline/space — kết hợp với `xargs -0` để xử lý an toàn tên file chứa khoảng trắng, tab, hay newline mà không bị tách nhầm.

🔑 **Từ khóa:** `find -print0` + `xargs -0` = cặp bài trùng xử lý tên file có khoảng trắng.

Câu 51. Lệnh xem hệ thống đã chạy bao lâu

Question: Which of the following commands can be used to determine how long the system has been running? (Choose two.) (Dịch câu hỏi: Lệnh nào xác định hệ thống đã chạy (uptime) bao lâu? (Chọn 2))

- A. uptime
- B. up
- C. time --up
- D. uname -u
- E. top

Dịch nghĩa các đáp án:

- A. `uptime`
- B. `up`
- C. `time --up`
- D. `uname -u`
- E. `top`

✅ **Answer: A, E**

Giải thích: `uptime` in trực tiếp thời gian hệ thống đã chạy, số user, load average. `top` cũng hiển thị dòng đầu tiên chứa thông tin uptime tương tự (giao diện theo dõi tiến trình thời gian thực). B, C, D là lệnh/option không tồn tại.

🔑 **Từ khóa:** `uptime` và dòng đầu của `top` để cho biết thời gian chạy hệ thống.

Câu 52. `ls > files` khi file `files` chưa tồn tại

Question: What is true regarding the command `ls > files` if `files` does not exist? (Dịch câu hỏi: Điều gì đúng khi chạy `ls > files` nếu `files` chưa tồn tại?)

- A. The output of `ls` is printed to the terminal
- B. `files` is created and contains the output of `ls`
- C. An error message is shown and `ls` is not executed
- D. The command `files` is executed and receives the output of `ls`
- E. Any output of `ls` is discarded

Dịch nghĩa các đáp án:

- A. Output của `ls` in ra terminal
- B. `files` được tạo, chứa output của `ls`
- C. Báo lỗi, `ls` không chạy
- D. Lệnh `files` được thực thi, nhận output của `ls`
- E. Output của `ls` bị bỏ đi

✅ **Answer: B — files is created and contains the output of ls**

Giải thích: Redirect `>` **tạo file mới** nếu chưa tồn tại (hoặc ghi đè nếu đã có), rồi ghi toàn bộ stdout của lệnh bên trái vào đó. Không cần file tồn tại trước.

🔑 **Từ khóa:** `>` = tạo mới hoặc ghi đè file; `>>` = tạo mới hoặc **nhập thêm** (append) vào cuối.

Câu 53. File chứa lịch sử Bash trong home directory

Question: Which of the following files, located in a user's home directory, contains the Bash history?

(Dịch câu hỏi: File nào trong home directory chứa lịch sử lệnh Bash?)

- A. `.bashrc_history`
- B. `.bash_histfile`
- C. `.history`
- D. `.bash_history`
- E. `.history_bash`

Dịch nghĩa các đáp án:

- A. `.bashrc_history`
- B. `.bash_histfile`
- C. `.history`
- D. `.bash_history`
- E. `.history_bash`

✅ **Answer: D — .bash_history**

Giải thích: Bash mặc định ghi lịch sử lệnh vào `~/.bash_history` (đường dẫn có thể đổi qua biến `$HISTFILE`), được ghi khi shell thoát (hoặc theo cấu hình `PROMPT_COMMAND/histappend`).

🔑 **Từ khóa:** `~/.bash_history` — tên file duy nhất đúng chính tả, các phương án khác chỉ là "gài bẫy" tên gần giống.

Câu 54. Wildcard khớp `ttyS0 ttyS1 ttyS2`

Question: Which wildcards will match the following filenames? (Choose two.) `ttyS0 ttyS1 ttyS2` (Dịch câu hỏi: Wildcard nào khớp các tên file `ttyS0 ttyS1 ttyS2`? (Chọn 2))

- A. `ttyS[1-5]`
- B. `tty?[0-5]`
- C. `tty*2`
- D. `ttyA-Z`
- E. `ttySs`

Dịch nghĩa các đáp án:

- A. `ttyS[1-5]`
- B. `tty?[0-5]`
- C. `tty*2`
- D. `tty[A-Z][012]`
- E. `tty[Ss][02]`

✅ **Answer: B, D**

Giải thích: `tty?[0-5]`: `?` khớp đúng 1 ký tự bất kỳ (chữ "S"), `[0-5]` khớp chữ số 0-5 → khớp cả `ttyS0`, `ttyS1`, `ttyS2`. `tty[A-Z][012]`: `[A-Z]` khớp chữ hoa "S", `[012]` khớp đúng 0,1,2 → khớp cả 3 file.

❌ A `ttyS[1-5]` chỉ khớp từ 1-5, **thiếu** `ttyS0`. ❌ C `tty*2` chỉ khớp file kết thúc bằng "2" → chỉ khớp `ttyS2`. ❌ E `tty[Ss][02]` chỉ khớp 0 và 2, **thiếu** `ttyS1`.

🔑 **Từ khóa:** Đọc kỹ range: `[1-5]` thiếu 0; `[02]` chỉ có 0 và 2 (không có 1) — bẫy kinh điển của LPIC.

Câu 55. Redirect output của `ls` sang standard error

Question: Which of the following commands redirects the output of `ls` to standard error? (Dịch câu hỏi: Lệnh nào redirect output của `ls` sang standard error?)

- A. `ls >-1`
- B. `ls <`
- C. `ls >&2`
- D. `ls >>2`
- E. `ls |error`

Dịch nghĩa các đáp án:

- A. `ls >-1`
- B. `ls <`
- C. `ls >&2`
- D. `ls >>2`

- E. `ls |error`

✅ **Answer: C — `ls >&2`**

Giải thích: `>&2` là cách viết tắt/tương đương của `1>&2` — redirect file descriptor 1 (stdout, mặc định) sang cùng đích với file descriptor 2 (stderr). Đây là kỹ thuật hay dùng để in thông báo lỗi ra đúng stream stderr.

🔑 **Từ khóa:** `>&2` (hoặc `1>&2`) = đẩy stdout ra chung với stderr.

Câu 56. Xem nội dung archive tar nén gzip

Question: Which of the following commands displays the contents of a gzip compressed tar archive? (Dịch câu hỏi: Lệnh nào hiển thị nội dung một archive tar nén gzip?)

- A. `gzip archive.tgz | tar xvf -`
- B. `tar -fzt archive.tgz`
- C. `gzip -d archive.tgz | tar tvf -`
- D. `tar cf archive.tgz`
- E. `tar ztf archive.tgz`

Dịch nghĩa các đáp án:

- A. `gzip archive.tgz | tar xvf -`
- B. `tar -fzt archive.tgz`
- C. `gzip -d archive.tgz | tar tvf -`
- D. `tar cf archive.tgz`
- E. `tar ztf archive.tgz`

✅ **Answer: E — `tar ztf archive.tgz`**

Giải thích: `tar` với option `z` (giải nén gzip), `t` (list — chỉ liệt kê, không giải nén ra đĩa), `f` (chỉ định tên file archive). Tổ hợp `ztf` = list nội dung archive .tgz mà không cần giải nén thủ công bằng `gzip` trước.

❌ B viết sai thứ tự tham số của `-f` (phải đứng ngay trước tên file). ❌ D `tar cf` là **tạo** (create) archive, không phải xem.

🔑 **Từ khóa:** tar: `c`=create, `x`=extract, `t`=list(view); `z`=gzip, `f`=file — nhớ `ztf` / `xzf` / `czf`.

Câu 57. In username và group ID chính từ /etc/passwd

Question: Which of the following commands prints a list of usernames (first column) and their primary group (fourth column) from the /etc/passwd file? (Dịch câu hỏi: Lệnh nào in username (cột 1) và group ID chính (cột 4) từ `/etc/passwd`?)

- A. `fmt -f 1,4 /etc/passwd`
- B. `cut -d : -f 1,4 /etc/passwd`
- C. `sort -t : -k 1,4 /etc/passwd`
- D. `paste -f 1,4 /etc/passwd`
- E. `split -c 1,4 /etc/passwd`

Dịch nghĩa các đáp án:

- A. `fmt -f 1,4 /etc/passwd`
- B. `cut -d : -f 1,4 /etc/passwd`
- C. `sort -t : -k 1,4 /etc/passwd`
- D. `paste -f 1,4 /etc/passwd`
- E. `split -c 1,4 /etc/passwd`

✅ **Answer: B — `cut -d : -f 1,4 /etc/passwd`**

Giải thích: `cut` trích xuất cột theo delimiter: `-d :` chỉ định dấu `:` là ký tự phân cách (đúng định dạng `/etc/passwd`), `-f 1,4` chọn cột 1 và cột 4. `sort` dùng để sắp xếp (không trích cột như yêu cầu), `fmt` / `paste` / `split` không có option `-f` để chọn cột kiểu này.

🔑 **Từ khóa:** `cut -d <ký_tự> -f <số_cột>` = công cụ chuẩn để trích cột theo delimiter.

Câu 58. Regex biểu diễn 1 chữ cái in hoa

Question: Which of the following regular expressions represents a single upper-case letter? (Dịch câu hỏi: Regex nào biểu diễn một chữ cái in hoa đơn lẻ?)

- A. `:UPPER:`
- B. `[A-Z]`
- C. `!a-z`
- D. `%C`
- E. `{AZ}`

Dịch nghĩa các đáp án:

- A. `:UPPER:`
- B. `[A-Z]`
- C. `!a-z`
- D. `%C`
- E. `{AZ}`

✅ **Answer: B — `[A-Z]`**

Giải thích: `[A-Z]` là cú pháp **character class** chuẩn trong regex POSIX, khớp đúng 1 ký tự nằm trong khoảng từ A đến Z (chữ hoa). Các đáp án khác không phải cú pháp regex hợp lệ.

 **Từ khóa:** `[A-Z]` = 1 chữ hoa; `[a-z]` = 1 chữ thường; `[0-9]` = 1 chữ số — cú pháp range trong ngoặc vuông.

Câu 59. [FILL BLANK] Lệnh chạy chương trình khác với nice level cho trước

Question: FILL BLANK - Which command is used to start another command with a given nice level? (Specify ONLY the command without any path or parameters.) (Dịch câu hỏi: Lệnh nào dùng để khởi động một chương trình khác với nice level chỉ định?)

 **Answer:** `nice`

Giải thích: `nice -n <giá_trị> <lệnh>` khởi chạy tiến trình **mới** với độ ưu tiên (niceness) chỉ định ngay từ đầu. Khác với `renice` dùng để đổi độ ưu tiên của tiến trình **đang chạy** (xem Câu 111).

 **Từ khóa:** `nice` = đặt priority khi **khởi chạy mới**; `renice` = đổi priority của tiến trình **đã chạy**.

Câu 60. Lọc log entries trong khoảng 8:00-8:59 bằng grep

Question: Given a log file loga.log with timestamps of the format DD/MM/YYYY:hh:mm:ss, which command filters out all log entries in the time period between 8:00 am and 8:59 am? (Dịch câu hỏi: File log `loga.log` có timestamp dạng `DD/MM/YYYY:hh:mm:ss`. Lệnh nào lọc các dòng trong khoảng 8:00 đến 8:59?)

- A. `grep -E ':08:[09]+:[09]+' loga.log`
- B. `grep -E ':08:[00]+' loga.log`
- C. `grep -E loga.log ':08:[0-9]+:[0-9]+'`
- D. `grep loga.log ':08:[0-9]:[0-9]'`
- E. `grep -E ':08:[0-9]+:[0-9]+' loga.log`

Dịch nghĩa các đáp án:

- A. `grep -E ':08:[09]+:[09]+' loga.log`
- B. `grep -E ':08:[00]+' loga.log`
- C. `grep -E loga.log ':08:[0-9]+:[0-9]+'`
- D. `grep loga.log ':08:[0-9]:[0-9]'`
- E. `grep -E ':08:[0-9]+:[0-9]+' loga.log`

 **Answer:** E — `grep -E ':08:[0-9]+:[0-9]+' loga.log`

Giải thích: Cần regex khớp mẫu `:08:` (giờ 08) theo sau bởi bất kỳ phút:giây nào — dùng `[0-9]+` (một hoặc nhiều chữ số) cho cả phút và giây. `-E` bật extended regex để `+` được hiểu là quantifier. Cú pháp `grep [options] 'pattern' file` — pattern **luôn đứng trước** tên file (loại C, D vì đảo ngược thứ tự).

❌ A dùng `[09]+` sai — đây là character class chỉ gồm ký tự 'o' hoặc '9', không phải "00-99".

🔑 **Từ khóa:** `grep -E 'pattern' file` (pattern trước, file sau); `[0-9]+` = 1+ chữ số bất kỳ; `[09]` ≠ dải số, chỉ là 2 ký tự 0 hoặc 9.

Câu 61. Xác định partition thay vì dùng tên device trong `/etc/fstab`

Question: Instead of supplying an explicit device in `/etc/fstab` for mounting, what other options may be used to identify the intended partition? (Choose two.) (Dịch câu hỏi: Thay vì ghi tên device tường minh trong `/etc/fstab`, tùy chọn nào khác có thể dùng để xác định partition? (Chọn 2))

- A. LABEL
- B. ID
- C. FIND
- D. NAME
- E. UUID

Dịch nghĩa các đáp án:

- A. LABEL
- B. ID
- C. FIND
- D. NAME
- E. UUID

✅ **Answer: A, E**

Giải thích: `/etc/fstab` cho phép xác định partition bằng **LABEL=** (nhãn filesystem do người dùng đặt) hoặc **UUID=** (định danh duy nhất tự sinh khi tạo filesystem) — cả hai bền vững hơn tên device (`/dev/sdaX` có thể đổi thứ tự giữa các lần boot).

🔑 **Từ khóa:** `fstab` ổn định hơn với `LABEL=` hoặc `UUID=` thay vì `/dev/sdaX` (dễ đổi tên khi thêm/bớt ổ đĩa).

Câu 62. Cài toàn bộ package group bằng yum

Question: A yum repository can declare sets of related packages. Which yum command installs all packages belonging to the group `admintools`? (Dịch câu hỏi: yum repo khai báo các nhóm gói liên quan. Lệnh nào cài tất cả gói thuộc nhóm `admintools`?)

- A. `yum pkgset --install admintools`
- B. `yum install admintools/*`
- C. `yum groupinstall admintools`

- **D.** yum taskinstall admintools
- **E.** yum collection install admintools

Dịch nghĩa các đáp án:

- **A.** `yum pkgset --install admintools`
- **B.** `yum install admintools/*`
- **C.** `yum groupinstall admintools`
- **D.** `yum taskinstall admintools`
- **E.** `yum collection install admintools`

✅ **Answer: C — yum groupinstall admintools**

Giải thích: yum hỗ trợ khái niệm **package groups** (tập hợp gói được gom nhóm sẵn theo mục đích, ví dụ "Development Tools"). `yum groupinstall <tên_nhóm>` cài toàn bộ gói trong nhóm đó cùng lúc.

🔑 **Từ khóa:** `yum groupinstall` = cài theo **nhóm** gói định sẵn (không phải cài từng gói lẻ).

Câu 63. [FILL BLANK] File cấu hình chính của yum ⚠️

Question: FILL BLANK - What directory contains configuration files for additional yum repositories? (Specify the full path to the directory.) (Dịch câu hỏi: Thư mục nào chứa các file cấu hình cho các repo yum bổ sung?)

✅ **Answer:** `/etc/yum.conf`

⚠️ **Lưu ý quan trọng:** Đây là câu hỏi hỏi về **thư mục** chứa cấu hình repo bổ sung — trên thực tế đáp án chuẩn xác về mặt kỹ thuật phải là `/etc/yum.repos.d/` (thư mục chứa các file `.repo`), còn `/etc/yum.conf` chỉ là **1 file** cấu hình chung của chính yum (không phải thư mục chứa "các repo bổ sung"). Đây có thể là lỗi trong ngân hàng đề — khi ôn thi thật, bạn nên nhớ **cả hai**:

- `/etc/yum.conf`: file cấu hình chính của yum.
- `/etc/yum.repos.d/`: thư mục chứa từng file `.repo` cho mỗi repository bổ sung.

🔑 **Từ khóa:** Nhớ cả bộ đôi: `yum.conf` (file, cấu hình chung) vs `yum.repos.d/` (thư mục, từng repo riêng).

Câu 64. Cài GRUB boot files vào partition đầu tiên của ổ đĩa đầu tiên

Question: Which of the following commands installs the GRUB boot files into the currently active file systems and the boot loader into the first partition of the first disk? (Dịch câu hỏi: Lệnh nào cài GRUB boot files vào filesystem đang active và bootloader vào partition đầu tiên của ổ đĩa đầu tiên?)

- A. grub-install /dev/sda
- B. grub-install /dev/sda1
- C. grub-install current /dev/sda0
- D. grub-install /dev/sda0
- E. grub-install current /dev/sda1

Dịch nghĩa các đáp án:

- A. `grub-install /dev/sda`
- B. `grub-install /dev/sda1`
- C. `grub-install current /dev/sda0`
- D. `grub-install /dev/sda0`
- E. `grub-install current /dev/sda1`

✅ **Answer: B — grub-install /dev/sda1**

Giải thích: Chỉ định **partition cụ thể** (`/dev/sda1` — partition số 1, tức đầu tiên) thay vì cả ổ đĩa (`/dev/sda`) khiến GRUB cài **boot sector của chính partition đó** (thường dùng khi partition đó được đánh dấu "boot" hoặc dùng chuỗi boot kiểu chainload). Lưu ý Linux đánh số partition bắt đầu từ 1, không có "sda0".

🔑 **Từ khóa:** `grub-install /dev/sdX` = cài vào MBR cả ổ; `grub-install /dev/sdXN` = cài vào chính partition N.

Câu 65. File tìm thấy trong /boot/

Question: Which of the following files are found in the /boot/ file system? (Choose two.) (Dịch câu hỏi: File nào có trong filesystem `/boot/`? (Chọn 2))

- A. Linux kernel images
- B. Bash shell binaries
- C. systemd target and service units
- D. Initial ramdisk images
- E. fsck binaries

Dịch nghĩa các đáp án:

- A. Linux kernel images
- B. Bash shell binaries
- C. systemd target/service units
- D. Initial ramdisk images
- E. fsck binaries

✅ **Answer: A, D**

Giải thích: `/boot/` chứa những gì cần thiết **ngay khi khởi động**, trước khi hệ thống chính được mount đầy đủ: file kernel (`vmlinuz-*`) và initramfs/initrd (`initramfs-*`, `initrd.img-*`), cùng cấu hình GRUB. Shell binaries (`/bin`), systemd units (`/etc/systemd`, `/usr/lib/systemd`), fsck binaries (`/sbin`) nằm ở nơi khác.

Từ khóa: `/boot/` = chỉ chứa kernel + initramfs + bootloader config, **KHÔNG** chứa binary chương trình.

Câu 66. File định nghĩa nguồn tải gói của Debian

Question: Which file defines the network locations from where the Debian package manager downloads software packages? (Dịch câu hỏi: File nào định nghĩa các địa chỉ mạng mà Debian package manager tải gói phần mềm về?)

- A. `/etc/dpkg/dpkg.cfg`
- B. `/etc/apt/apt.conf.d`
- C. `/etc/apt/apt.conf`
- D. `/etc/dpkg/dselect.cfg`
- E. `/etc/apt/sources.list`

Dịch nghĩa các đáp án:

- A. `/etc/dpkg/dpkg.cfg`
- B. `/etc/apt/apt.conf.d`
- C. `/etc/apt/apt.conf`
- D. `/etc/dpkg/dselect.cfg`
- E. `/etc/apt/sources.list`

✅ **Answer: E — `/etc/apt/sources.list`**

Giải thích: `/etc/apt/sources.list` (và các file bổ sung trong `/etc/apt/sources.list.d/`) liệt kê các **địa chỉ repository** (URL, distro, component) mà `apt` / `apt-get` dùng để tải gói. Đừng nhầm với `apt.conf` — file đó là cấu hình **hành vi** của apt (proxy, timeout...), không phải danh sách nguồn tải.

Từ khóa: `sources.list` = **nguồn tải gói** (repo URL); `apt.conf` = **cấu hình hành vi** apt.

Câu 67. Option dpkg gỡ package kèm cả file cấu hình

Question: When removing a package on a system using dpkg package management, which dpkg option ensures configuration files are removed as well? (Dịch câu hỏi: Option `dpkg` nào đảm bảo file cấu hình cũng bị xóa khi gỡ gói?)

- A. `--clean`
- B. `--purge`

- C. `--vacuum`
- D. `--remove`
- E. `--declare`

Dịch nghĩa các đáp án:

- A. `--clean`
- B. `--purge`
- C. `--vacuum`
- D. `--remove`
- E. `--declare`

✅ **Answer: B — `--purge`**

Giải thích: Xem lại Câu 33: `--remove` / `-r` chỉ gỡ binary, giữ config; `--purge` / `-P` gỡ **toàn bộ**, kể cả file cấu hình còn sót lại trong `/etc/`.

🔑 **Từ khóa:** `purge` = gỡ "sạch sẽ" (kể cả config); `remove` = gỡ nhưng giữ config.

Câu 68. So sánh Container vs Virtual Machine

Question: Which of the following statements are correct when comparing Linux containers with traditional virtual machines (e.g. LXC vs. KVM)? (Choose three.) (Dịch câu hỏi: Phát biểu nào đúng khi so sánh Linux container với VM truyền thống (LXC vs KVM)? (Chọn 3))

- A. Containers are a lightweight virtualization method where the kernel controls process isolation and resource management.
- B. Fully virtualized machines can run any operating system for a specific hardware architecture within the virtual machine.
- C. Containers are completely decoupled from the host system's physical hardware and can only use emulated virtual hardware devices.
- D. The guest environment for fully virtualized machines is created by a hypervisor which provides virtual and emulated hardware devices.
- E. Containers on the same host can use different operating systems, as the container hypervisor creates separate kernel execution.

Dịch nghĩa các đáp án:

- A. Container là ảo hóa nhẹ, kernel quản lý cô lập tiến trình và tài nguyên
- B. VM đầy đủ có thể chạy bất kỳ OS nào phù hợp kiến trúc phần cứng
- C. Container hoàn toàn tách biệt khỏi phần cứng vật lý host, chỉ dùng thiết bị ảo hóa mô phỏng (sai — container **chia sẻ** kernel/phần cứng host)
- D. Môi trường guest của VM đầy đủ được tạo bởi hypervisor cung cấp thiết bị ảo/mô phỏng
- E. Container trên cùng host có thể chạy OS khác nhau vì "container hypervisor" tạo kernel riêng (sai — container **dùng chung 1 kernel** của host)

✅ **Answer: A, B, D**

Giải thích: Điểm khác biệt cốt lõi: Container **chia sẻ kernel** của host (nhẹ, khởi động nhanh, cô lập ở mức process/namespace/cgroup) — nên **không thể** chạy OS khác kernel Linux (loại C, E). VM đầy đủ (KVM/hypervisor) mô phỏng phần cứng ảo hoàn chỉnh, cho phép chạy **bất kỳ OS** nào (kể cả Windows) trên cùng phần cứng vật lý.

🔑 **Từ khóa:** Container = chung kernel host (nhẹ); VM = hypervisor + phần cứng ảo, mỗi VM có kernel/OS riêng (nặng nhưng linh hoạt OS).

Câu 69. Sửa lỗi cài gói Debian local do thiếu dependency

Question: The installation of a local Debian package failed due to unsatisfied dependencies. Which of the following commands installs missing dependencies and completes the interrupted package installation? (*Dịch câu hỏi: Cài gói Debian local thất bại do thiếu dependency. Lệnh nào cài dependency còn thiếu và hoàn tất cài đặt bị gián đoạn?*)

- A. `dpkg --fix --all`
- B. `apt-get autoinstall`
- C. `dpkg-reconfigure --all`
- D. `apt-get all`
- E. `apt-get install -f`

Dịch nghĩa các đáp án:

- A. `dpkg --fix --all`
- B. `apt-get autoinstall`
- C. `dpkg-reconfigure --all`
- D. `apt-get all`
- E. `apt-get install -f`

✅ **Answer: E — apt-get install -f**

Giải thích: `-f` (`--fix-broken`) yêu cầu `apt-get` tự động tìm và cài các gói phụ thuộc còn thiếu để "chữa" các gói đang ở trạng thái cài dở/gãy (broken), thường dùng ngay sau khi `dpkg -i file.deb` báo lỗi thiếu dependency.

🔑 **Từ khóa:** `apt-get install -f` = "fix broken" — cứu cánh kinh điển sau khi `dpkg -i` báo lỗi dependency.

Câu 70. Liệt kê tất cả gói đã cài (RPM)

Question: Which of the following commands lists all currently installed packages when using RPM package management? (*Dịch câu hỏi: Lệnh nào liệt kê tất cả gói đã cài đặt khi dùng RPM package management?*)

- A. yum --query --all
- B. yum --list --installed
- C. rpm --query --list
- D. rpm --list --installed
- E. rpm --query --all

Dịch nghĩa các đáp án:

- A. `yum --query --all`
- B. `yum --list --installed`
- C. `rpm --query --list`
- D. `rpm --list --installed`
- E. `rpm --query --all`

✅ **Answer: E — rpm --query --all**

Giải thích: `rpm -qa` (query all) liệt kê toàn bộ gói đã cài trên hệ thống. `rpm --query --list (-ql)` khác hẳn — nó liệt kê **các file bên trong một gói cụ thể**, không phải danh sách gói.

🔑 **Từ khóa:** `rpm -qa` = list mọi gói đã cài; `rpm -ql <pkg>` = list file **bên trong** 1 gói cụ thể — dễ nhầm hai option này.

Câu 71. Lệnh hợp lệ trong file cấu hình GRUB 2

Question: Which of the following commands are valid in the GRUB 2 configuration file? (Choose two.)
(Dịch câu hỏi: Lệnh nào hợp lệ trong file cấu hình GRUB 2? (Chọn 2))

- A. menuentry
- B. uefi
- C. pxe-ifconfig
- D. insmod
- E. kpartx

Dịch nghĩa các đáp án:

- A. `menuentry`
- B. `uefi`
- C. `pxe-ifconfig`
- D. `insmod`
- E. `kpartx`

✅ **Answer: A, D**

Giải thích: `grub.cfg` là một dạng script riêng của GRUB: `menuentry "Tên" { ... }` định nghĩa mỗi mục trong boot menu; `insmod <module>` nạp module GRUB (ví dụ hỗ trợ đọc LVM, filesystem...) trước khi dùng. `uefi`, `pxe-ifconfig`, `kpartx` không phải lệnh GRUB.

 **Từ khóa:** `grub.cfg` dùng `menuentry {...}` (mỗi mục boot) + `insmod` (nạp module GRUB, KHÔNG phải module kernel).

Câu 72. Mục đích của lệnh `ldd`

Question: What is the purpose of the `ldd` command? (*Dịch câu hỏi: Mục đích của lệnh `ldd` là gì?*)

- **A.** It lists which shared libraries a binary needs to run.
- **B.** It installs and updates installed shared libraries.
- **C.** It turns a dynamically linked binary into a static binary.
- **D.** It defines which version of a library should be used by default.
- **E.** It runs a binary with an alternate library search path.

Dịch nghĩa các đáp án:

- **A.** Liệt kê các shared library mà 1 binary cần để chạy
- **B.** Cài đặt và cập nhật shared library
- **C.** Chuyển binary dynamically-linked thành static
- **D.** Định nghĩa version thư viện nào được dùng mặc định
- **E.** Chạy binary với đường dẫn tìm thư viện thay thế

 **Answer: A — It lists which shared libraries a binary needs to run.**

Giải thích: `ldd <binary>` in ra danh sách các thư viện chia sẻ (`.so`) mà chương trình phụ thuộc vào, kèm đường dẫn đã được resolve — rất hữu ích để debug lỗi "error while loading shared libraries".

 **Từ khóa:** `ldd` = list dynamic dependencies (thư viện `.so` cần thiết).

Câu 73. Công dụng của LVM

Question: What can the Logical Volume Manager (LVM) be used for? (Choose three.) (*Dịch câu hỏi: LVM (Logical Volume Manager) dùng để làm gì? (Chọn 3)*)

- **A.** To create snapshots.
- **B.** To dynamically change the size of logical volumes.
- **C.** To dynamically create or delete logical volumes.
- **D.** To create RAID 9 arrays.
- **E.** To encrypt logical volumes.

Dịch nghĩa các đáp án:

- **A.** Tạo snapshot
- **B.** Đổi kích thước logical volume linh động
- **C.** Tạo/xóa logical volume linh động

- **D.** Tạo RAID 9 (không tồn tại RAID level 9)
- **E.** Mã hóa logical volume (mã hóa là chức năng của LUKS/dm-crypt, không phải LVM)

✅ **Answer: A, B, C**

Giải thích: LVM cho phép: (1) tạo **snapshot** — bản chụp trạng thái tại 1 thời điểm, hữu ích cho backup; (2) **resize** volume linh hoạt (mở rộng/thu nhỏ) mà không cần unmount toàn hệ thống; (3) **tạo/xóa** logical volume tùy ý từ volume group, linh hoạt hơn phân vùng cứng truyền thống.

❌ **D** — "RAID 9" không tồn tại (chuẩn RAID chỉ có 0,1,4,5,6,10...). ❌ **E** — mã hóa là việc của LUKS/dm-crypt, LVM không tự mã hóa dữ liệu.

🔑 **Từ khóa:** LVM = resize + snapshot + tạo/xóa volume linh hoạt. LVM **không** mã hóa, **không** làm RAID (đó là mdadm/dm-crypt riêng).

Câu 74. Khác biệt chính giữa GPT và MBR

Question: What are the main differences between GPT and MBR partition tables regarding maximum number and size of partitions? (Choose two.) (*Dịch câu hỏi: Khác biệt chính giữa GPT và MBR về số lượng và kích thước partition tối đa? (Chọn 2)*)

- **A.** MBR can handle partition sizes up to 4 TB, whereas GPT supports partition sizes up to 128 ZB.
- **B.** By default, GPT can manage up to 128 partitions while MBR only supports four primary partitions.
- **C.** By default, GPT can manage up to 64 partitions while MBR only supports 16 primary partitions.
- **D.** MBR can handle partition sizes up to 2.2 TB, whereas GPT supports sizes up to 9.4 ZB.
- **E.** Both GPT and MBR support up to four primary partitions, each with up to 4096 TB.

Dịch nghĩa các đáp án:

- **A.** MBR tới 4TB, GPT tới 128ZB (sai số liệu)
- **B.** Mặc định GPT quản lý tới 128 partition, MBR chỉ hỗ trợ 4 partition chính (primary)
- **C.** Mặc định GPT tới 64 partition, MBR tới 16 primary (sai số liệu)
- **D.** MBR tới 2.2TB, GPT tới 9.4ZB
- **E.** Cả GPT và MBR hỗ trợ tối đa 4 primary partition, mỗi cái 4096TB (sai)

✅ **Answer: B, D**

Giải thích: Đây là 2 giới hạn kinh điển cần thuộc:

- **Số lượng partition:** MBR giới hạn cứng **4 primary partition** (có thể lách bằng extended+logical); GPT hỗ trợ tới **128 partition** mặc định (không cần extended partition).
- **Kích thước:** MBR dùng địa chỉ 32-bit LBA → giới hạn khoảng **2.2 TB**; GPT dùng địa chỉ 64-bit LBA → lý thuyết tới **9.4 ZB** (zettabyte).

🔑 **Từ khóa:** MBR: 4 primary, ~2.2TB. GPT: 128 partition, ~9.4ZB. Cần nhớ đúng 2 cặp số này.

Câu 75. Lợi ích của hard link trong backup software

Question: A backup software heavily uses hard links between files which have not been changed in between two backup runs. Which benefits are realized due to these hard links? (Choose two.) (*Dịch câu hỏi: Backup software dùng hard link nhiều giữa các file không đổi qua 2 lần backup. Lợi ích nào đạt được? (Chọn 2))*)

- **A.** The old backups can be moved to slow backup media, such as tapes, while still serving as hard link target in new backups.
- **B.** The backup runs faster because hard links are asynchronous operations, postponing the copy operation to a later point in time.
- **C.** The backup is guaranteed to be uncharged because a hard linked file cannot be modified after its creation.
- **D.** The backup consumes less space because the hard links point to the same data on disk instead of storing redundant copies.
- **E.** The backup runs faster because, instead of copying the data of each file, hard links only change file system meta data.

Dịch nghĩa các đáp án:

- **A.** Backup cũ chuyển sang media chậm (tape) vẫn giữ được vai trò hard link target (sai — hard link cần cùng filesystem, không hoạt động qua tape)
- **B.** Backup nhanh hơn vì hard link là thao tác bất đồng bộ (sai — không liên quan đồng bộ/bất đồng bộ)
- **C.** Backup đảm bảo không đổi vì file hard link không thể sửa sau khi tạo (sai — hard link vẫn có thể bị sửa nội dung)
- **D.** Backup tốn ít dung lượng hơn vì hard link trỏ cùng dữ liệu trên đĩa thay vì lưu bản sao dư thừa
- **E.** Backup nhanh hơn vì thay vì copy dữ liệu từng file, hard link chỉ thay đổi metadata filesystem

✅ **Answer: D, E**

Giải thích: Kỹ thuật "hard-link backup" (như `rsync --link-dest`) tạo hard link tới file **không thay đổi** từ bản backup trước thay vì copy lại toàn bộ — vừa **tiết kiệm dung lượng** (D, không có 2 bản dữ liệu vật lý) vừa **nhanh hơn nhiều** (E, chỉ tạo entry metadata trỏ inode, không copy nội dung thật).

🔑 **Từ khóa:** Hard-link backup = tiết kiệm không gian + tiết kiệm thời gian, vì chỉ thao tác metadata, không copy dữ liệu thật.

Câu 76. [FILL BLANK] File /proc liệt kê thiết bị đang mount

Question: FILL BLANK - Which file from the /proc/ file system contains a list of all currently mounted devices? (Specify the full name of the file, including path.) (*Dịch câu hỏi: File nào trong /proc/ chứa danh sách tất cả thiết bị đang mount?*)

✅ **Answer:** `/proc/mounts`

Giải thích: `/proc/mounts` (symlink tới `/proc/self/mounts`) hiển thị danh sách mount hiện tại ở định dạng giống `/etc/fstab`, luôn phản ánh trạng thái **thực tế** đang chạy (không giống `/etc/fstab` chỉ là cấu hình dự định).

 **Từ khóa:** `/proc/mounts` = trạng thái mount **thực tế** hiện tại; `/etc/fstab` = cấu hình **dự định** khi boot.

Câu 77. Số trường (field) trong 1 dòng `/etc/fstab` hợp lệ

Question: How many fields are in a syntactically correct line of `/etc/fstab`? (Dịch câu hỏi: Một dòng hợp lệ trong `/etc/fstab` có bao nhiêu trường (field)?)

- A. 3
- B. 4
- C. 5
- D. 6
- E. 7

Dịch nghĩa các đáp án:

- A. 3
- B. 4
- C. 5
- D. 6
- E. 7

 **Answer: D — 6**

Giải thích: Mỗi dòng `/etc/fstab` gồm đúng 6 cột: **device** — **mountpoint** — **filesystemtype** — **options** — **dump** — **pass** (ví dụ: `UUID=... /home ext4 defaults 0 2`).

 **Từ khóa:** `fstab` = 6 cột: `<device> <mount_point> <type> <options> <dump> <pass>`.

Câu 78. Lý do `chmod 640` không cập nhật quyền

Question: Running `chmod 640 filea.txt` as a regular user doesn't update `filea.txt`'s permission. What might be a reason why `chmod` cannot modify the permissions? (Choose two.) (Dịch câu hỏi: Chạy `chmod 640 filea.txt` với quyền user thường nhưng không cập nhật được. Lý do có thể là gì? (Chọn 2))

- A. `filea.txt` is owned by another user and a regular user cannot change the permissions of another user's file.
- B. `filea.txt` is a symbolic link whose permissions are a fixed value which cannot be changed.
- C. `filea.txt` has the sticky bit set and a regular user cannot remove this permission.
- D. `filea.txt` is a hard link whose permissions are inherited from the target and cannot be set directly.

- **E.** filea.txt has the SetUID bit set which imposes the restriction that only the root user can make changes to the file.

Dịch nghĩa các đáp án:

- **A.** filea.txt thuộc sở hữu user khác, user thường không thể đổi quyền file người khác
- **B.** filea.txt là symbolic link có quyền cố định không thể đổi
- **C.** filea.txt có sticky bit, user thường không gỡ được (sai — sticky bit không ngăn owner chmod file của chính mình)
- **D.** filea.txt là hard link, quyền kế thừa từ target, không set trực tiếp được (sai — hard link cùng inode nên chmod vẫn tác động trực tiếp)
- **E.** filea.txt có SetUID, chỉ root mới sửa được (sai — SetUID không ngăn owner chmod)

✅ **Answer: A, B**

Giải thích:

- **A:** Chỉ **owner** (hoặc root) mới có quyền chmod một file. Nếu user thường không phải chủ sở hữu → lệnh thất bại (Operation not permitted).
- **B:** Trên Linux, quyền hiển thị của symbolic link (lrwxrwxrwx) thường **cố định** (không có ý nghĩa thực), chmod trên symlink thực ra thường tác động tới **target** — nhưng nếu hệ thống không hỗ trợ đổi quyền symlink, lệnh không thay đổi gì đáng kể trên bản thân symlink.

🔑 **Từ khóa:** chmod chỉ owner/root làm được; quyền symlink thường không ý nghĩa/không đổi trực tiếp.

Câu 79. Filesystem Linux cấp phát trước số lượng inode cố định

Question: Which of the following Linux filesystems preallocate a fixed number of inodes when creating a new filesystem instead of generating them as needed? (Choose two.) (*Dịch câu hỏi: Filesystem Linux nào cấp phát trước (preallocate) một số lượng inode cố định khi tạo mới, thay vì sinh theo nhu cầu? (Chọn 2)*)

- **A.** JFS
- **B.** ext3
- **C.** XFS
- **D.** ext2
- **E.** procfs

Dịch nghĩa các đáp án:

- **A.** JFS
- **B.** ext3
- **C.** XFS
- **D.** ext2

- E. procs

✅ **Answer: B, D**

Giải thích: Họ **ext (ext2/ext3/ext4)** dùng cơ chế cấp phát **cố định số lượng inode ngay khi format** (dựa trên kích thước fs và tỷ lệ bytes-per-inode) — nên có thể "hết inode" dù còn trống dung lượng đĩa. Ngược lại, các filesystem hiện đại như **XFS, JFS, Btrfs** cấp phát inode **động** theo nhu cầu thực tế, không giới hạn cứng từ đầu.

🔑 **Từ khóa:** Họ ext (2/3/4) = inode cố định từ lúc format (có thể hết inode dù còn dung lượng); XFS/JFS/Btrfs = inode động.

Câu 80. Lệnh set quyền SetUID cho file thực thi

Question: Which of the following commands sets the SetUID permission on the executable /bin/foo?
(Dịch câu hỏi: Lệnh nào set quyền SetUID cho file thực thi `/bin/foo`?)

- A. `chmod 4755 /bin/foo`
- B. `chmod 1755 /bin/foo`
- C. `chmod u-s /bin/foo`
- D. `chmod 755+s /bin/foo`
- E. `chmod 2755 /bin/foo`

Dịch nghĩa các đáp án:

- A. `chmod 4755 /bin/foo`
- B. `chmod 1755 /bin/foo`
- C. `chmod u-s /bin/foo`
- D. `chmod 755+s /bin/foo`
- E. `chmod 2755 /bin/foo`

✅ **Answer: A — `chmod 4755 /bin/foo`**

Giải thích: Bit đặc biệt (special bit) đứng đầu trong ký hiệu octal 4 chữ số: **4** = SetUID, **2** = SetGID, **1** = Sticky bit. Vậy `4755` = SetUID + `rw-r-xr-x`.

❌ B `1755` là **sticky bit**, không phải SetUID. ❌ E `2755` là **SetGID**. ❌ C `u-s` là **gỡ bỏ SetUID** (dấu trừ), không phải thêm vào.

🔑 **Từ khóa:** Bit đặc biệt: 4=SUID, 2=SGID, 1=Sticky — nhớ theo thứ tự "4-2-1" giống cách tính quyền `rwx` (4=r,2=w,1=x) nhưng ở "tầng" đặc biệt.

Câu 81. Lệnh hiển thị số inode của 1 file

Question: Which of the following commands can be used to display the inode number of a given file?
(Choose two.) (Dịch câu hỏi: Lệnh nào hiển thị số inode của 1 file? (Chọn 2))

- A. inode
- B. ln
- C. ls
- D. cp
- E. stat

Dịch nghĩa các đáp án:

- A. `inode`
- B. `ln`
- C. `ls`
- D. `cp`
- E. `stat`

✅ **Answer: C, E**

Giải thích: `ls -li` hiển thị số inode kèm tên file (cột đầu tiên). `stat <file>` hiển thị thông tin chi tiết về file, bao gồm cả trường "Inode". `inode` không phải lệnh có thật.

🔑 **Từ khóa:** `ls -li` và `stat` để xem được inode number.

Câu 82. umask cho quyền file mặc định `-rw-r-----`

Question: Which of the following settings for umask ensures that new files have the default permissions `-rw-r-----`? (Dịch câu hỏi: Giá trị umask nào đảm bảo file mới có quyền mặc định `-rw-r-----`?)

- A. 0017
- B. 0640
- C. 0038
- D. 0227
- E. 0027

Dịch nghĩa các đáp án:

- A. 0017
- B. 0640
- C. 0038
- D. 0227
- E. 0027

✅ **Answer: E — 0027**

Giải thích: File mới mặc định bắt đầu từ 666 (rw-rw-rw-, không có x). Muốn kết quả `rw-r-----` (640): $666 - 640 = 026...$ nhưng thực ra công thức đúng theo bit AND-NOT: umask áp dụng bit nào KHÔNG được set. Kiểm tra trực tiếp: umask 027 → owner giữ nguyên (6 & ~0=6=rw-), group: $6 \& \sim 2 = 4$ (r--), other: $6 \& \sim 7 = 0$ (---) → kết quả `rw-r-----` ✅. Đây cùng giá trị umask **0027** như Câu 2 (dùng chung cho cả file và thư mục, chỉ khác base 666 vs 777).

🔑 **Từ khóa:** umask 0027 dùng chung mọi lúc: file → `rw-r-----`, thư mục → `rwxr-x---`.

Câu 83. Sửa lỗi filesystem XFS bị inconsistent sau mất điện

Question: After a power outage, the XFS file system of `/dev/sda3` is inconsistent. How can the existing file system errors be fixed? (Dịch câu hỏi: Sau mất điện, filesystem XFS trên `/dev/sda3` bị lỗi. Làm sao sửa các lỗi filesystem?)

- A. By using mount -f to force a mount of the file system
- B. By running xfsck on the file system
- C. By mounting the file system with the option xfs_repair
- D. By running xfsadmin repair on the file system
- E. By running xfs_repair on the file system

Dịch nghĩa các đáp án:

- A. `mount -f` để force mount
- B. Chạy `xfsck` (không tồn tại lệnh này)
- C. Mount với option `xfs_repair` (sai — không phải mount option)
- D. Chạy `xfsadmin repair` (lệnh bịa)
- E. Chạy `xfs_repair`

✅ **Answer: E — By running xfs_repair on the file system**

Giải thích: XFS không dùng `fsck` kiểu truyền thống như ext (`e2fsck`) — công cụ chuyên dụng để kiểm tra/sửa lỗi XFS là `xfs_repair`, chạy khi filesystem đã **unmount**. Lưu ý: XFS tự động "replay log" khi mount lại sau crash, nhưng nếu lỗi nghiêm trọng hơn thì cần `xfs_repair` thủ công.

🔑 **Từ khóa:** ext* → `e2fsck` / `fsck.ext4`; XFS → `xfs_repair` (không có "xfsck").

Câu 84. Thuộc tính file thay đổi khi tạo hard link trở tới nó

Question: Which of the following properties of an existing file changes when a hard link pointing to that file is created? (Dịch câu hỏi: Thuộc tính nào của 1 file đã tồn tại thay đổi khi tạo 1 hard link trở tới nó?)

- A. File size
- B. Link count
- C. Modify timestamp
- D. Inode number
- E. Permissions

Dịch nghĩa các đáp án:

- A. Kích thước file

- **B.** Link count
- **C.** Modify timestamp
- **D.** Số inode
- **E.** Quyên hạn

✅ **Answer: B — Link count**

Giải thích: Vì hard link chỉ là thêm 1 "tên" nữa trỏ tới cùng inode, nên duy nhất **link count** (số liên kết cứng trỏ tới inode đó, xem qua `ls -l` cột thứ 2) sẽ **tăng thêm 1**. Kích thước, nội dung, timestamp sửa đổi, quyền, và số inode đều **không đổi** vì đó là cùng 1 dữ liệu vật lý.

🔑 **Từ khóa:** Tạo hard link → chỉ đổi **link count** (+1), mọi thứ khác của file gốc giữ nguyên.

Câu 85. [FILL BLANK] Vị trí đặt binary tự biên dịch theo FHS

Question: FILL BLANK - Following the Filesystem Hierarchy Standard (FHS), where should binaries that have been compiled by the system administrator be placed in order to be made available to all users on the system? (Specify the full path to the directory.) (*Dịch câu hỏi: Theo FHS, binary do sysadmin tự biên dịch nên đặt ở đâu để mọi user trên hệ thống dùng được?*)

✅ **Answer:** `/usr/local/bin/`

Giải thích: FHS dành riêng `/usr/local/` cho phần mềm **cài đặt cục bộ bởi sysadmin** (không qua package manager của distro) — tách biệt để tránh xung đột/bị ghi đè khi distro cập nhật gói hệ thống. `/usr/local/bin/` là nơi đặt binary trong nhánh đó.

🔑 **Từ khóa:** `/usr/local/` = "lãnh địa riêng" của sysadmin, an toàn trước update hệ thống qua package manager.

Câu 86. Lệnh xem shell xử lý 1 command cụ thể như thế nào

Question: Which of the following commands show how the shell handles a specific command? (*Dịch câu hỏi: Lệnh nào cho biết shell xử lý một lệnh cụ thể như thế nào (alias, builtin, file...)?*)

- **A.** where
- **B.** type
- **C.** stat
- **D.** case
- **E.** fileinfo

Dịch nghĩa các đáp án:

- **A.** `where`
- **B.** `type`
- **C.** `stat`

- D. `case`
- E. `fileinfo`

✅ **Answer: B — type**

Giải thích: `type <lệnh>` cho biết lệnh đó là: **builtin** (lệnh có sẵn của shell), **alias**, **function**, hay **file thực thi** (kèm đường dẫn) — hữu ích hơn `which` vì `which` chỉ tìm trong `$PATH`, không biết được builtin/alias.

🔑 **Từ khóa:** `type` = biết lệnh là builtin/alias/function/file; `which` = chỉ tìm file trong `$PATH`.

Câu 87. Ký tự bắt đầu tìm kiếm ngược trong vi (Normal mode)

Question: When in Normal mode in vi, which character can be used to begin a reverse search of the text? (Dịch câu hỏi: Ở chế độ Normal của vi, ký tự nào bắt đầu tìm kiếm **ngược** (reverse search)?)

- A. r
- B. /
- C. F
- D. ?
- E. s

Dịch nghĩa các đáp án:

- A. `r`
- B. `/`
- C. `F`
- D. `?`
- E. `s`

✅ **Answer: D — ?**

Giải thích: `/pattern` tìm **xuôi** (forward, tìm về phía cuối file); `?pattern` tìm **ngược** (backward, tìm về phía đầu file). `n` lặp lại tìm kiếm cùng hướng, `N` lặp lại theo hướng ngược lại.

🔑 **Từ khóa:** `/` = tìm xuôi (forward); `?` = tìm ngược (backward) — như dấu gạch nghiêng "xuôi" và dấu hỏi "ngược".

Câu 88. Xem man page của lệnh ở section 1

Question: Which of the following commands displays the manual page command from section 1? (Dịch câu hỏi: Lệnh nào hiển thị man page của `command` từ section 1?)

- A. `man command(1)`
- B. `man command@1`

- C. `man 1 command`
- D. `man 1.command`
- E. `man -s 1 command`

Dịch nghĩa các đáp án:

- A. `man command(1)`
- B. `man command@1`
- C. `man 1 command`
- D. `man 1.command`
- E. `man -s 1 command`

✅ **Answer: C — `man 1 command`**

Giải thích: Cú pháp chuẩn để chỉ định section man page: `man <section> <command>` — số section đặt trước tên lệnh. Man page chia thành nhiều section (1=lệnh user, 2=system calls, 3=library calls, 5=file formats...), hữu ích khi 1 tên trùng ở nhiều section (VD `man 5 passwd` khác `man 1 passwd` ... thực ra `passwd` ở section 1 và 5).

🔑 **Từ khóa:** `man <số_section> <tên_lệnh>` — số đứng trước, không có option `-s` chuẩn POSIX cho việc này.

Câu 89. Tạo/ghi đè file với output của `ls`

Question: Which of the following commands creates or, in case it already exists, overwrites a file called `data` with the output of `ls`? (Dịch câu hỏi: Lệnh nào tạo mới (hoặc ghi đè nếu đã tồn tại) file `data` với output của `ls`?)

- A. `ls 3> data`
- B. `ls >& data`
- C. `ls > data`
- D. `ls >> data`
- E. `ls >>> data`

Dịch nghĩa các đáp án:

- A. `ls 3> data`
- B. `ls >& data`
- C. `ls > data`
- D. `ls >> data`
- E. `ls >>> data`

✅ **Answer: C — `ls > data`**

Giải thích: `>` = tạo mới hoặc **ghi đè** hoàn toàn. `>>` = tạo mới hoặc **nhớ i thêm** (append) vào cuối nếu đã tồn tại (không ghi đè). Đây là 2 toán tử khác nhau rõ ràng cần phân biệt.

 **Từ khóa:** `>` = ghi đè; `>>` = nối thêm — luôn nhớ số dấu `>` càng nhiều thì "an toàn" hơn (append, không mất dữ liệu cũ).

Câu 90. Lệnh đổi option và positional parameter trong Bash đang chạy

Question: Which of the following commands is used to change options and positional parameters within a running Bash shell? (Dịch câu hỏi: Lệnh nào dùng để đổi option và positional parameter trong 1 phiên Bash đang chạy?)

- A. history
- B. setsh
- C. bashconf
- D. set
- E. envsetup

Dịch nghĩa các đáp án:

- A. `history`
- B. `setsh`
- C. `bashconf`
- D. `set`
- E. `envsetup`

✅ **Answer: D — set**

Giải thích: `set` là builtin của Bash dùng để bật/tắt shell option (VD `set -x` bật debug trace, `set -e` dừng khi lỗi) và/hoặc gán lại positional parameters (`$1, $2...`). Đây là công cụ cấu hình hành vi shell ngay trong phiên hiện tại.

 **Từ khóa:** `set -x` / `set -e` ... = bật/tắt tùy chọn shell (shell option) ngay lập tức, không cần khởi động lại.

Câu 91. Hiển thị PID của tất cả tiến trình thuộc root

Question: Which of the following commands display the IDs of all processes owned by root? (Choose two.) (Dịch câu hỏi: Lệnh nào hiển thị ID của tất cả tiến trình thuộc user root? (Chọn 2))

- A. `pgrep -c root`
- B. `pgrep -u root`
- C. `pgrep -f root`
- D. `pgrep -U o`
- E. `pgrep -c o`

Dịch nghĩa các đáp án:

- A. `pgrep -c root`
- B. `pgrep -u root`
- C. `pgrep -f root`
- D. `pgrep -U 0`
- E. `pgrep -c 0`

✅ **Answer: B, D**

Giải thích: `pgrep -u <user>` lọc theo **tên hoặc UID** của effective user — cả `-u root` (tên) và `-U 0` (UID số, vì root luôn có UID=0) đều hợp lệ và cho cùng kết quả. `-c` chỉ **đếm** số lượng (không in PID), `-f` lọc theo full command line (không phải theo owner).

🔑 **Từ khóa:** root ⇔ UID 0; `pgrep -u` / `-U` = lọc theo user (tên/UID); `-c` chỉ đếm, không liệt kê PID.

Câu 92. Chuỗi lệnh vi lưu và thoát

Question: Which of the following sequences in the vi editor saves the opened document and exits the editor? (Choose two.) (Dịch câu hỏi: Chuỗi lệnh nào trong vi lưu file đang mở và thoát editor? (Chọn 2))

- A. Ctrl XX
- B. Ctrl :W
- C. Esc zz
- D. Esc :wq
- E. Esc ZZ

Dịch nghĩa các đáp án:

- A. `Ctrl XX`
- B. `Ctrl :W`
- C. `Esc zz`
- D. `Esc :wq`
- E. `Esc ZZ`

✅ **Answer: D, E**

Giải thích: Từ Insert mode, nhấn `Esc` để về Normal mode trước, sau đó:

- `:wq` (Enter) — lệnh mode: **w**rite (lưu) + **q**uit (thoát).
- `ZZ` (viết hoa, không cần Enter, không cần dấu `:`) — phím tắt Normal mode tương đương lưu+thoát.

🔑 **Từ khóa:** `:wq` hoặc `ZZ` (hoa) đều = lưu + thoát; `zz` (thường) chỉ là lệnh cuộn màn hình, khác hẳn.

Câu 93. Tác dụng của option `-v` trong `grep`

Question: What is the effect of the `-v` option for the `grep` command? (Dịch câu hỏi: Tác dụng của option `-v` trong lệnh `grep` là gì?)

- A. It enables color to highlight matching parts.
- B. It shows the command's version information.
- C. It only outputs non-matching lines.
- D. It changes the output order showing the last matching line first.
- E. It outputs all lines and prefixes matching lines with `a+`.

Dịch nghĩa các đáp án:

- A. Bật màu highlight phần khớp
- B. Hiện version của lệnh
- C. Chỉ in các dòng KHÔNG khớp
- D. Đảo thứ tự in dòng khớp cuối trước
- E. In tất cả dòng, đánh dấu dòng khớp bằng dấu `a+`

✅ **Answer: C — It only outputs non-matching lines.**

Giải thích: `grep -v pattern` đảo ngược logic khớp — chỉ hiển thị các dòng **KHÔNG** chứa pattern (invert match). Rất hữu ích để lọc bỏ (loại trừ) các dòng không mong muốn, ví dụ `grep -v '^#'` loại bỏ dòng comment.

🔑 **Từ khóa:** `-v` = invert match = chỉ in dòng **KHÔNG** khớp.

Câu 94. Công cụ hiển thị đường dẫn đầy đủ của file thực thi shell sẽ chạy

Question: Which of the following tools can show the complete path of an executable file that the current shell would execute when starting a command without specifying its complete path? (Choose two.) (Dịch câu hỏi: Công cụ nào hiển thị đường dẫn đầy đủ của file thực thi mà shell hiện tại sẽ chạy khi gõ 1 lệnh không kèm đường dẫn? (Chọn 2))

- A. find
- B. pwd
- C. which
- D. locate
- E. type

Dịch nghĩa các đáp án:

- A. `find`
- B. `pwd`
- C. `which`

- D. `locate`
- E. `type`

✅ **Answer: C, E**

Giải thích: Cả `which` và `type` đều tìm được **đường dẫn thực thi** dựa trên `$PATH` (xem lại Câu 49, 86) — `type` còn thông minh hơn khi nhận diện cả builtin/alias. `find`, `locate` tìm theo tên file toàn hệ thống, không quan tâm `$PATH`; `pwd` chỉ in thư mục hiện tại.

🔑 **Từ khóa:** Tìm lệnh "sẽ chạy khi gõ" → luôn nghĩ tới `which` hoặc `type` (dựa trên `$PATH`), không phải `find` / `locate`.

Câu 95. Tín hiệu gửi khi nhấn Ctrl+Z

Question: Which of the following signals is sent to a process when the key combination Ctrl+Z is pressed on the keyboard? (*Dịch câu hỏi: Tín hiệu nào gửi tới tiến trình khi nhấn Ctrl+Z?*)

- A. SIGTERM
- B. SIGCONT
- C. SIGSTOP
- D. SIGKILL
- E. SIGINT

Dịch nghĩa các đáp án:

- A. SIGTERM
- B. SIGCONT
- C. SIGSTOP
- D. SIGKILL
- E. SIGINT

✅ **Answer: C — SIGSTOP**

Giải thích: Ctrl+Z gửi tín hiệu **tạm dừng** tiến trình (thực chất là SIGTSTP, có thể bắt/xử lý được — đề bài chọn gần đúng gọi chung nhóm "STOP"), đưa tiến trình về trạng thái "Stopped" (chạy nên tạm dừng), khác hẳn Ctrl+C (SIGINT, xem Câu 46) là yêu cầu **dừng hẳn**. Dùng `fg` / `bg` để tiếp tục tiến trình đã dừng.

🔑 **Từ khóa:** Ctrl+Z = tạm dừng (SIGSTOP/SIGTSTP) → `fg` / `bg` để chạy tiếp; Ctrl+C = SIGINT (ngắt hẳn).

Câu 96. Kết quả regex `s/[ABC] [abc]/xx/` áp dụng cho "ABCabc"

Question: What is the output when the regular expression `s/[ABC] [abc]/xx/` is applied to the following string? ABCabc - (Dịch câu hỏi: Output khi áp dụng regex `s/[ABC] [abc]/xx/` cho chuỗi `ABCabc`?)

- A. ABxxbc
- B. xxCxxc
- C. xxxxxx
- D. ABCabc
- E. Axxaxx

Dịch nghĩa các đáp án:

- A. `ABabc`
- B. `xxCxxc`
- C. `xxxxxx`
- D. `ABCabc`
- E. `Axxaxx`

✅ **Answer: A — ABxxbc**

Giải thích: Regex `[ABC] [abc]` yêu cầu: 1 ký tự trong {A,B,C}, theo sau bởi 1 khoảng trắng (space), rồi 1 ký tự trong {a,b,c}. Chuỗi `ABCabc` không hề có khoảng trắng nào giữa các ký tự → pattern không khớp ở đâu cả → chuỗi giữ nguyên, không có gì được thay thế.

🔑 **Từ khóa:** Bẫy kinh điển: khoảng trắng trong `[ABC] [abc]` là ký tự literal cần khớp — không có nghĩa là "hoặc". Không có space trong input → không match.

Câu 97. Lệnh in thư mục làm việc hiện tại trong Bash

Question: Which of the following commands print the current working directory when using a Bash shell? (Choose two.) (Dịch câu hỏi: Lệnh nào in thư mục làm việc hiện tại khi dùng Bash shell? (Chọn 2))

- A. `echo "${PWD}"`
- B. `echo "${WD}"`
- C. `printwd`
- D. `pwd`
- E. `echo "${pwd}"`

Dịch nghĩa các đáp án:

- A. `echo "${PWD}"`
- B. `echo "${WD}"`
- C. `printwd`
- D. `pwd`

- E. `echo "${pwd}"`

✅ **Answer: A, D**

Giải thích: `pwd` (print working directory) là lệnh builtin chuẩn. Bash cũng tự động duy trì biến môi trường `$PWD` (chú ý viết HOA toàn bộ — biến shell phân biệt hoa/thường) luôn chứa thư mục hiện tại, nên `echo "${PWD}"` cho cùng kết quả.

❌ E dùng biến `pwd` (chữ thường) — **không tồn tại** vì biến shell phân biệt chữ hoa/thường.

🔑 **Từ khóa:** `$PWD` (biến, viết HOA) ⇔ lệnh `pwd` — cả hai cho cùng kết quả thư mục hiện tại.

Câu 98. Lệnh in ra chữ "test" bằng heredoc

Question: Which of the following commands outputs test to the shell? (*Dịch câu hỏi: Lệnh nào xuất ra chữ "test" ra shell?*)

- A. `cat test EOT -`
- B. `cat <|EOT test EOT -`
- C. `cat ! test EOT -`
- D. `cat & test EOT -`
- E. `cat < test EOT -`

Dịch nghĩa các đáp án:

- A. `cattestEOT -`
- B. `cat <|EOTtestEOT -`
- C. `cat !testEOT -`
- D. `cat &testEOT -`
- E. `cat <testEOT -`

✅ **Answer: E — cat < test EOT -**

Giải thích: Đây là cú pháp **heredoc**: `<<DELIM ... nội dung ... DELIM`. Format hiển thị trong đề bị dính liền do copy từ web (thực tế là `cat << EOT` xuống dòng `test` xuống dòng `EOT`). Trong các lựa chọn, chỉ E có cấu trúc gần đúng dùng `<` (input redirect/heredoc) hợp lý về mặt cú pháp so với các ký tự bị khác (`|`, `!`, `&`).

🔑 **Từ khóa:** Heredoc cú pháp: `cat << EOT` ← nội dung ← `EOT` — dùng để nhúng văn bản nhiều dòng ngay trong script.

Câu 99. Nice level mặc định khi khởi động tiến trình bằng `nic`

e

Question: What is the default nice level when a process is started using the nice command? (Dịch câu hỏi: Nice level mặc định khi khởi động 1 tiến trình bằng lệnh `nice` (không kèm option)?)

- A. -10
- B. 0
- C. 10
- D. 15
- E. 20

Dịch nghĩa các đáp án:

- A. -10
- B. 0
- C. 10
- D. 15
- E. 20

✓ **Answer: C — 10**

Giải thích: Nếu gọi `nice <lệnh>` mà không chỉ định `-n <giá_trị>`, mặc định GNU `nice` sẽ cộng thêm 10 vào nice level hiện tại (thường từ 0 lên 10) — nghĩa là tiến trình chạy "nhường nhịn hơn một chút" so với mặc định, chứ không phải giữ nguyên 0.

🔑 **Từ khóa:** `nice` (không tham số) = mặc định +10 vào nice level, không phải 0.

Câu 100. Xóa thư mục có tên chứa ký tự ``

Question: A user accidentally created the subdirectory `\dir` in his home directory. Which of the following commands will remove that directory? (Dịch câu hỏi: User lỡ tạo thư mục `\dir` trong home directory. Lệnh nào xóa được thư mục đó?)

- A. `rmdir ~/\dir`
- B. `rmdir "~/\dir"`
- C. `rmdir ~/'dir'`
- D. `rmdir ~/\dir`
- E. `rmdir '~/\dir'`

Dịch nghĩa các đáp án:

- A. `rmdir ~/\dir`
- B. `rmdir "~/\dir"`
- C. `rmdir ~/'dir'`
- D. `rmdir ~/\dir`

- E. `rmdir '~/\dir'`

✅ **Answer: A — `rmdir ~/\dir`**

Giải thích: Trong shell, dấu ``` là ký tự **escape**. Muốn shell hiểu ``` là ký tự literal (một phần của tên thư mục thật), phải escape nó thành `\``. Cũng lưu ý `~` (dấu ngã, home directory) chỉ được **mở rộng (expand)** khi đứng **ngoài** dấu nháy — nếu đặt trong nháy đơn (`'~/...'`) thì `~` sẽ bị hiểu là ký tự literal, không trở về home directory nữa (loại C, E vì `~` bị "vô hiệu hóa" trong nháy đơn).

🔑 **Từ khóa:** ``` trong tên file cần escape thành `\``; `~` chỉ expand khi KHÔNG bị bao trong nháy đơn.

Câu 101. Lệnh tìm kiếm bằng regex trong nội dung file

Question: Which of the following commands can perform searches on file contents using regular expressions? (Dịch câu hỏi: Lệnh nào có thể tìm kiếm trong nội dung file bằng biểu thức chính quy (regex)?)

- A. find
- B. locate
- C. grep
- D. reggrep
- E. pgrep

Dịch nghĩa các đáp án:

- A. `find`
- B. `locate`
- C. `grep`
- D. `reggrep`
- E. `pgrep`

✅ **Answer: C — `grep`**

Giải thích: `grep` là công cụ chuyên tìm chuỗi/pattern **bên trong nội dung** file bằng regex. `find` tìm theo thuộc tính file (tên, thời gian, quyền...) chứ không đọc nội dung. `locate` tìm theo tên file trong database. `pgrep` tìm tiến trình đang chạy, không phải nội dung file. `reggrep` không tồn tại.

🔑 **Từ khóa:** `grep` = tìm bên trong **nội dung** file; `find` = tìm theo **thuộc tính** file (tên/size/date...).

Câu 102. Option `find` giới hạn số cấp thư mục con

Question: In a nested directory structure, which find command line option would be used to restrict the command to searching down a particular number of subdirectories? (Dịch câu hỏi: Trong cấu trúc thư mục lồng nhau, option nào của `find` giới hạn tìm kiếm theo số cấp thư mục con?)

- A. -maxdepth
- B. -dirmax
- C. -maxlevels
- D. -s
- E. -n

Dịch nghĩa các đáp án:

- A. `-maxdepth`
- B. `-dirmax`
- C. `-maxlevels`
- D. `-s`
- E. `-n`

✅ **Answer: A — -maxdepth**

Giải thích: `find <path> -maxdepth <N>` giới hạn `find` chỉ đi sâu tới đa N cấp thư mục con tính từ điểm bắt đầu — hữu ích khi không muốn quét toàn bộ cây thư mục quá sâu. Có option đối lập `-mindepth` (giới hạn tối thiểu).

🔑 **Từ khóa:** `find -maxdepth N` = giới hạn độ sâu tìm kiếm; đi cùng cặp với `-mindepth`.

Câu 103. Lệnh xác định định dạng file bằng database "magic"

Question: Which of the following commands determines a file's format by using a definition database file which contains information about all common file types? (Dịch câu hỏi: Lệnh nào xác định định dạng (format) của 1 file bằng cách dùng database định nghĩa chứa thông tin các loại file phổ biến?)

- A. type
- B. file
- C. magic
- D. pmagic
- E. hash

Dịch nghĩa các đáp án:

- A. `type`
- B. `file`
- C. `magic`
- D. `pmagic`
- E. `hash`

✅ **Answer: B — file**

Giải thích: Lệnh `file <tên_file>` đọc vài byte đầu (magic number/magic bytes) của file và đối chiếu với database định nghĩa (`/usr/share/misc/magic` hoặc tương tự) để xác định đúng loại file (JPEG, ELF binary, ASCII text...) — **không** phụ thuộc vào phần mở rộng tên file, khác với đoán qua đuôi

file.

 **Từ khóa:** `file` = nhận diện loại file dựa trên **magic bytes** (nội dung thật), không dựa vào tên/đuôi file.

Câu 104. Lấy username kèm login shell từ `/etc/passwd`

Question: Which of the following commands generates a list of user names from `/etc/passwd` along with their login shell? (Dịch câu hỏi: Lệnh nào tạo danh sách username kèm login shell từ `/etc/passwd`?)

- A. `column -s : 1,7 /etc/passwd`
- B. `chop -c 1,7 /etc/passwd`
- C. `colrm 1,7 /etc/passwd`
- D. `sort -t: -k1,7 /etc/passwd`
- E. `cut -d: -f1,7 /etc/passwd`

Dịch nghĩa các đáp án:

- A. `column -s : 1,7 /etc/passwd`
- B. `chop -c 1,7 /etc/passwd`
- C. `colrm 1,7 /etc/passwd`
- D. `sort -t: -k1,7 /etc/passwd`
- E. `cut -d: -f1,7 /etc/passwd`

 **Answer: E — `cut -d: -f1,7 /etc/passwd`**

Giải thích: Trong `/etc/passwd`, cột 1 = username, cột 7 = login shell (theo thứ tự: username:password:UID:GID:GECOS:home:shell). `cut -d: -f1,7` trích đúng 2 cột đó, dùng dấu `:` làm delimiter — tương tự cách dùng ở Câu 57 nhưng chọn cột khác.

 **Từ khóa:** `/etc/passwd` có 7 trường: user:pass:uid:gid:gecos:home:shell — cột 1 và cột 7 là username và shell.

Câu 105. Kết quả sau khi `gunzip` file `.tgz` chứa 2 file

Question: If the gzip compressed tar archive `texts.tgz` contains the files `a.txt` and `b.txt`, which files will be present in the current directory after running `gunzip texts.tgz`? (Dịch câu hỏi: Archive `texts.tgz` (gzip nén tar) chứa `a.txt` và `b.txt`. Sau khi chạy `gunzip texts.tgz`, những file nào tồn tại trong thư mục hiện tại?)

- A. Only `a.txt`, `b.txt`, and `texts.tgz`
- B. Only `texts.tar` and `texts.tgz`
- C. Only `a.txt.gz` and `b.txt.gz`

- D. Only a.txt and b.txt
- E. Only texts.tar

Dịch nghĩa các đáp án:

- A. Chỉ `a.txt, b.txt, texts.tgz`
- B. Chỉ `texts.tar, texts.tgz`
- C. Chỉ `a.txt.gz, b.txt.gz`
- D. Chỉ `a.txt, b.txt`
- E. Chỉ `texts.tar`

✅ **Answer: E — Only texts.tar**

Giải thích: `gunzip` chỉ giải nén tệp **ng gzip** (bỏ nén, đổi `.tgz` / `.tar.gz` thành `.tar`), **không** tự động giải nén tệp **ng tar** (chưa "extract" các file bên trong archive). Muốn lấy được `a.txt`, `b.txt` thật sự phải chạy tiếp `tar xf texts.tar`, hoặc dùng thẳng `tar xzf texts.tgz` để làm cả 2 bước cùng lúc.

🔑 **Từ khóa:** `.tgz` = 2 tệp: nén (gzip) + đóng gói (tar). `gunzip` chỉ gỡ tệp nén → ra `.tar`, chưa ra file gốc.

Câu 106. Cách lặp lại lệnh vi nhiều lần (số lần trước lệnh)

Question: In the vi editor, how can commands such as moving the cursor or copying lines into the buffer be issued multiple times or applied to multiple rows? (Dịch câu hỏi: Trong vi, làm sao lệnh (di chuyển con trỏ, copy dòng...) được thực hiện nhiều lần hoặc áp dụng cho nhiều dòng?)

- A. By using the command `:repeat` followed by the number and the command
- B. By specifying the number right in front of a command such as `4j` or `2yj`.
- C. By selecting all affected lines using the shift and cursor keys before applying the command.
- D. By issuing a command such as `:set repetition=4` with repeats every subsequent command 4 times.
- E. By specifying the number after a command such as `14` or `yj2` followed by escape.

Dịch nghĩa các đáp án:

- A. Dùng lệnh `:repeat` kèm số và lệnh
- B. Ghi số ngay trước lệnh, ví dụ `4j` hoặc `2yj`
- C. Chọn các dòng bằng Shift + phím mũi tên trước khi áp dụng lệnh
- D. Dùng `:set repetition=4`
- E. Ghi số sau lệnh, ví dụ `14` hoặc `yj2` rồi Esc

✅ **Answer: B — By specifying the number right in front of a command such as 4j or 2yj.**

Giải thích: Đây chính là quy tắc **số** đã nhắc ở Câu 38: số nhân lặp luôn đặt **trước** lệnh trong vi, ví dụ `4j` (xuống 4 dòng), `2yj` (copy dòng hiện tại + dòng kế, tổng 2 dòng). Không có `:repeat`, `:set repetition` trong vi thật.

🔑 **Từ khóa:** vi: số đặt **trước** lệnh (không phải sau) để lặp lại/áp dụng nhiều dòng.

Câu 107. Ý nghĩa dấu `&` cuối câu lệnh

Question: Which of the following statements is correct for a command ending with an `&` character?

(Dịch câu hỏi: Phát biểu nào đúng khi 1 lệnh kết thúc bằng ký tự `&`?)

- A. The command's output is redirected to `/dev/null`.
- B. The command is run in background of the current shell.
- C. The command's output is executed by the shell.
- D. The command is run as a direct child of the init process.
- E. The command's input is read from `/dev/null`.

Dịch nghĩa các đáp án:

- A. Output redirect vào `/dev/null`
- B. Lệnh chạy nền (background) của shell hiện tại
- C. Output của lệnh được shell thực thi (vô nghĩa)
- D. Lệnh chạy như con trực tiếp của init
- E. Input đọc từ `/dev/null`

✅ **Answer: B — The command is run in background of the current shell.**

Giải thích: Dấu `&` ở cuối câu lệnh yêu cầu shell chạy lệnh đó ở **chế độ nền (background)** — shell trả lại quyền điều khiển ngay cho người dùng (không đợi lệnh hoàn thành), có thể theo dõi bằng `jobs`, đưa lên foreground bằng `fg`.

🔑 **Từ khóa:** `&` cuối lệnh = chạy nền (background job), không phải redirect hay input null.

Câu 108. Lệnh đọc file và chia thành nhiều phần (chunk) theo kích thước

Question: Which of the following commands reads a file and creates separate chunks of a given size from the file's contents? (Dịch câu hỏi: Lệnh nào đọc 1 file và tạo ra các "chunk" riêng biệt theo kích thước cho trước từ nội dung file?)

- A. `ar`
- B. `cat`
- C. `break`
- D. `split`
- E. `parted`

Dịch nghĩa các đáp án:

- A. `ar`
- B. `cat`
- C. `break`
- D. `split`

- E. `parted`

✅ **Answer: D — `split`**

Giải thích: `split -b <size> file prefix` chia 1 file lớn thành nhiều file nhỏ hơn theo kích thước chỉ định (ví dụ chia file 1GB thành các phần 100MB để gửi qua email hoặc USB dung lượng nhỏ). Có thể ghép lại bằng `cat part* > original`.

🔑 **Từ khóa:** `split` = chia file lớn thành nhiều file nhỏ theo size; ghép lại bằng `cat`.

Câu 109. Mục đích của lệnh `xargs`

Question: What is the purpose of the `xargs` command? (*Dịch câu hỏi: Mục đích của lệnh `xargs` là gì?*)

- A. It passes arguments to an X server.
- B. It repeats the execution of a command using different parameters for each invocation.
- C. It reads standard input and builds up commands to execute.
- D. It asks a question, graphically, and returns the answer to the shell.
- E. It allows specifying long options (like `--help`) for commands that normally only accept short options (like `-h`)

Dịch nghĩa các đáp án:

- A. Truyền tham số cho X server
- B. Lặp lại thực thi 1 lệnh với tham số khác nhau mỗi lần (mô tả gần đúng nhưng chưa đầy đủ bản chất)
- C. Đọc standard input và xây dựng, thực thi các lệnh
- D. Hỏi 1 câu hỏi đồ họa, trả lời về shell
- E. Cho phép dùng long option (như `--help`) cho lệnh chỉ nhận short option

✅ **Answer: C — It reads standard input and builds up commands to execute.**

Giải thích: `xargs` đọc dữ liệu từ **standard input** (thường là output của 1 lệnh khác qua pipe, ví dụ `find ... | xargs rm`), rồi **xây dựng và thực thi** lệnh chỉ định kèm các tham số lấy từ input đó — giải quyết vấn đề nhiều lệnh (như `rm`, `find`) không đọc trực tiếp từ stdin mà cần tham số dòng lệnh.

🔑 **Từ khóa:** `xargs` = "câu nô i" biến stdin thành **tham số dòng lệnh** cho lệnh khác thực thi.

Câu 110. [FILL BLANK] Lệnh hiển thị danh sách background task

Question: FILL BLANK - Which command displays a list of all background tasks running in the current shell? (Specify ONLY the command without any path or parameters.) (*Dịch câu hỏi: Lệnh nào hiển thị danh sách tất cả background task đang chạy trong shell hiện tại?*)

✓ **Answer:** `jobs`

Giải thích: `jobs` liệt kê các tiến trình được shell hiện tại quản lý ở chế độ nền/tạm dừng, kèm số job (`[1]`, `[2]` ...) dùng với `fg %1`, `bg %1`, `kill %1`.

🔑 **Từ khóa:** `jobs` = xem danh sách job nền của **shell hiện tại** (khác `ps` xem toàn bộ tiến trình hệ thống).

Câu 111. [FILL BLANK] Lệnh đổi độ ưu tiên tiến trình đang chạy

Question: FILL BLANK - Which command is used to change the priority of an already running process? (Specify ONLY the command without any path or parameters.) (Dịch câu hỏi: Lệnh nào dùng để đổi độ ưu tiên (priority) của 1 tiến trình đang chạy?)

✓ **Answer:** `renice`

Giải thích: Xem lại Câu 59: `nice` chỉ dùng khi **khởi động mới** tiến trình; `renice <giá_trị> -p <PID>` mới là lệnh đổi độ ưu tiên của tiến trình **đã đang chạy**.

🔑 **Từ khóa:** `nice` = lúc khởi động; `renice` = lúc đang chạy (đổi sau).

Câu 112. Ý nghĩa của `1>&2` trong Bash

Question: In Bash, inserting `1>&2` after a command redirects... (Dịch câu hỏi: Trong Bash, chèn `1>&2` sau 1 lệnh sẽ redirect...?)

- A. ...standard error to standard input.
- B. ...standard output to standard error.
- C. ...standard input to standard error.
- D. ...standard error to standard output.
- E. ...standard output to standard input.

Dịch nghĩa các đáp án:

- A. stderr sang stdin
- B. stdout sang stderr
- C. stdin sang stderr
- D. stderr sang stdout
- E. stdout sang stdin

✓ **Answer:** B — ...standard output to standard error.

Giải thích: `1>&2` đọc là "file descriptor 1 (stdout) được redirect đến (cùng đích với) file descriptor 2 (stderr)". Đây chính là cú pháp đầy đủ của `>&2` đã gặp ở Câu 55 — dùng để in thông báo (vốn định ra stdout) thành đi ra đúng luồng lỗi (stderr).

 **Từ khóa:** `1>&2` = đẩy stdout → stderr; `2>&1` (ngược lại) = đẩy stderr → stdout. Thứ tự số quyết định chiều redirect.

Câu 113. Debug hệ thống hang khi boot bằng rescue CD

Question: When booting from the hard disk, a computer successfully loads the Linux kernel and initramfs but hangs during the subsequent startup tasks. The system is booted using a Linux based rescue CD to investigate the problem. Which of the following methods helps to identify the root cause of the problem? (Dịch câu hỏi: Máy load kernel/initramfs thành công nhưng bị treo (hang) ở bước tiếp theo. Dùng rescue CD để điều tra. Phương pháp nào giúp xác định nguyên nhân gốc rễ?)

- **A.** Using the dmesg command from the rescue CD's shell to view the original system's boot logs.
- **B.** Investigating the file /proc/kmsg on the computer's hard disk for possible errors.
- **C.** Investigating the file /var/log on the computer's hard disk for possible errors.
- **D.** Using chroot to switch to the file system on the hard disk and use dmesg to view the logs.
- **E.** Rebooting again from the hard drive since the system successfully booted from the rescue CD.

Dịch nghĩa các đáp án:

- **A.** Dùng `dmesg` từ shell của rescue CD để xem boot log của hệ thống gốc (sai — dmesg của rescue CD chỉ log của chính rescue CD, không phải hệ thống gốc)
- **B.** Kiểm tra `/proc/kmsg` trên ổ cứng máy tính (sai — /proc/kmsg là filesystem ảo runtime, không lưu trên đĩa)
- **C.** Kiểm tra `/var/log` trên ổ cứng máy tính để tìm lỗi
- **D.** Dùng `chroot` để chuyển vào filesystem trên ổ cứng và dùng `dmesg` xem log
- **E.** Boot lại từ ổ cứng vì hệ thống đã boot thành công từ rescue CD (không liên quan tới việc sửa lỗi)

 **Answer: C — Investigating the file /var/log on the computer's hard disk for possible errors.**

Giải thích: Vì hệ thống gốc đã **hang trước khi ghi được log runtime** (dmesg/kmsg đều là bộ nhớ tạm của lần boot **hiện tại từ rescue CD**, không chứa dữ liệu của lần boot lỗi), nguồn thông tin đáng tin cậy nhất còn sót lại là **log đã ghi ra đĩa** ở những lần boot/chạy trước đó, tức `/var/log/` (syslog, boot.log...) trên ổ cứng máy tính — mount thủ công (không cần chroot) rồi xem trực tiếp các file log này.

 **D — `chroot` rồi `dmesg` không hữu ích vì `dmesg` (dù chroot) vẫn đọc ring buffer của kernel đang chạy (kernel rescue CD), không phải kernel/log của lần boot bị lỗi.**

 **Từ khóa:** dmesg/kmsg = log **runtime của kernel đang chạy hiện tại** (rescue CD), không "xuyên" được vào lần boot lỗi trước đó → phải xem log **đã ghi ra đĩa** (`/var/log`).

Câu 114. Vị trí lưu bootloader trên ổ đĩa hệ thống UEFI

Question: Where is the bootloader stored on the hard disk of a UEFI system? (*Dịch câu hỏi: Bootloader được lưu ở đâu trên ổ cứng của hệ thống UEFI?*)

- A. In the EFI Boot Record (EBR).
- B. In the Master Boot Record (MBR).
- C. On the EFI System Partition (ESP).
- D. On the partition labeled boot.
- E. On the partition number 127.

Dịch nghĩa các đáp án:

- A. EFI Boot Record (EBR) — không tồn tại khái niệm này
- B. Master Boot Record (MBR) — đó là kiểu BIOS/legacy
- C. EFI System Partition (ESP)
- D. Partition có label "boot" (không bắt buộc, không phải quy định UEFI)
- E. Partition số 127 (bịa)

✅ **Answer: C — On the EFI System Partition (ESP).**

Giải thích: Nhắc lại từ Câu 24: hệ UEFI lưu bootloader (file `.efi`) trên **ESP** — 1 partition FAT32 riêng biệt, khác hoàn toàn với cách BIOS lưu bootloader ngay trong 446 byte đầu của MBR.

🔑 **Từ khóa:** UEFI → bootloader nằm trong **ESP** (partition FAT32 riêng); BIOS → bootloader nằm trong **MBR** (sector đầu ổ đĩa).

Câu 115. Đặt default boot target của systemd thành multi-user

Question: What is the correct way to set the default systemd boot target to multi-user? (*Dịch câu hỏi: Cách đúng để đặt default systemd boot target thành `multi-user`?*)

- A. `systemctl isolate multi-user.target`
- B. `systemctl set-runlevel multi-user.target`
- C. `systemctl set-boot multi-user.target`
- D. `systemctl set-default multi-user.target`
- E. `systemctl boot -p multi-user.target`

Dịch nghĩa các đáp án:

- A. `systemctl isolate multi-user.target`
- B. `systemctl set-runlevel multi-user.target`
- C. `systemctl set-boot multi-user.target`
- D. `systemctl set-default multi-user.target`
- E. `systemctl boot -p multi-user.target`

✅ **Answer: D — systemctl set-default multi-user.target**

Giải thích: `systemctl set-default <target>` ghi cấu hình **vĩnh viễn** (tạo symlink `/etc/systemd/system/default.target` trỏ tới target chỉ định) — đây là target sẽ dùng ở **mọi lần boot sau này**.

❌ A `systemctl isolate` chỉ chuyển target ngay lập tức cho **phiên hiện tại**, không lưu lại cho lần boot sau — dễ nhầm với D vì đề cập liên quan target nhưng tác dụng khác nhau (tạm thời vs vĩnh viễn).

🔑 **Từ khóa:** `isolate` = đổi target **ngay bây giờ** (tạm thời); `set-default` = đổi target **mặc định mãi mãi** (khi boot).

Câu 116. Phát biểu đúng về initial RAM disk (initramfs)

Question: Which of the following statements are correct about the initial RAM disk involved in the boot process of Linux? (Choose two.) (Dịch câu hỏi: Phát biểu nào đúng về initial RAM disk trong quá trình boot Linux? (Chọn 2))

- A. An initramfs is a compressed file system archive, which can be unpacked to examine its contents.
- B. An initramfs file contains the MBR, the bootloader and the Linux kernel.
- C. After a successful boot, the initramfs contents are available in `/run/initramfs/`.
- D. The kernel uses the initramfs temporarily before accessing the real root file system.
- E. An initramfs does not depend on a specific kernel version and is not changed after the initial installation.

Dịch nghĩa các đáp án:

- A. initramfs là 1 archive filesystem nén, có thể unpack để xem nội dung
- B. File initramfs chứa MBR, bootloader và kernel Linux (sai — initramfs chỉ chứa driver/tool tạm, không chứa MBR/bootloader/kernel)
- C. Sau khi boot thành công, nội dung initramfs khả dụng ở `/run/initramfs/` (một phần đúng nhưng không phải điểm chính được hỏi — đáp án chuẩn hơn là D)
- D. Kernel dùng initramfs tạm thời trước khi truy cập root filesystem thật
- E. initramfs không phụ thuộc phiên bản kernel cụ thể và không đổi sau khi cài đặt lần đầu (sai — initramfs thường được sinh lại mỗi khi kernel/module thay đổi)

✅ **Answer: A, D**

Giải thích: initramfs là 1 archive (thường định dạng cpio nén gzip) chứa 1 filesystem tạm i gian với driver/module cần thiết (VD driver ổ đĩa, LVM, mã hóa) để kernel có thể **tìm và mount root filesystem thật** — sau đó kernel chuyển quyền điều khiển sang root fs thật (`pivot_root` / `switch_root`) và initramfs hoàn thành nhiệm vụ tạm thời của nó (D). Vì là archive nén thông thường, có thể unpack bằng `cpio` / `zcat` để xem nội dung (A).

🔑 **Từ khóa:** initramfs = filesystem tạm nén (cpio+gzip) chứa driver cần thiết → giúp kernel mount root fs thật, rồi bị thay thế (không dùng lại).

Câu 117. Lệnh nạp kernel module kèm dependency cần thiết

Question: Which of the following commands loads a kernel module along with any required dependency modules? (Dịch câu hỏi: Lệnh nào nạp 1 kernel module kèm theo mọi module phụ thuộc cần thiết?)

- A. depmod
- B. modprobe
- C. module_install
- D. insmod
- E. loadmod

Dịch nghĩa các đáp án:

- A. `depmod`
- B. `modprobe`
- C. `module_install`
- D. `insmod`
- E. `loadmod`

✅ **Answer: B — modprobe**

Giải thích: `modprobe <module>` thông minh hơn `insmod`: nó tự động tra cứu **dependency** (module nào cần module khác) và nạp **theo đúng thứ tự** tất cả các module liên quan. `insmod` chỉ nạp đúng 1 module chỉ định, nếu thiếu u dependency sẽ báo lỗi. `depmod` chỉ dùng để **sinh** file dependency (`module.s.dep`), không tự nạp module.

🔑 **Từ khóa:** `insmod` = nạp 1 module (không tự lo dependency); `modprobe` = nạp thông minh, tự kéo theo dependency.

Câu 118. Thông tin `lspci` có thể hiển thị

Question: What information can the lspci command display about the system hardware? (Choose three.) (Dịch câu hỏi: `lspci` có thể hiển thị thông tin gì về phần cứng hệ thống? (Chọn 3))

- A. System battery type
- B. Device IRQ settings
- C. PCI bus speed
- D. Ethernet MAC address
- E. Device vendor identification

Dịch nghĩa các đáp án:

- A. Loại pin hệ thống
- B. Cài đặt IRQ của thiết bị
- C. Tốc độ bus PCI

- **D.** Địa chỉ MAC Ethernet
- **E.** Thông tin nhận dạng nhà sản xuất thiết bị (vendor ID)

✅ **Answer: B, C, E**

Giải thích: `lspci` liệt kê chi tiết mọi thiết bị gắn trên bus PCI/PCIe: vendor/device ID (E), tốc độ bus (link speed, C), tài nguyên hệ thống được cấp cho thiết bị như IRQ, vùng địa chỉ I/O/memory (B).

❌ A "loại pin" và ❌ D "địa chỉ MAC" không phải thông tin cấp **bus PCI** — MAC address thuộc về driver mạng (`ip link` / `ifconfig`), pin thuộc ACPI (`acpi` / `upower`), không phải phạm vi của `lspci`.

🔑 **Từ khóa:** `lspci` = thông tin thiết bị **trên bus PCI** (vendor, IRQ, bus speed) — không phải MAC address hay pin.

Câu 119. [FILL BLANK] File cấu hình System V đặt default runlevel

Question: FILL BLANK - Which System V init configuration file is commonly used to set the default run level? (Specify the full name of the file, including path.) (Dịch câu hỏi: File cấu hình System V init nào thường dùng để đặt default runlevel?)

✅ **Answer:** `/etc/inittab`

Giải thích: Trên hệ System V init truyền thống (không phải systemd/Upstart), file `/etc/inittab` chứa dòng như `id:3:initdefault:` để chỉ định runlevel mặc định (0-6) khi boot — tương đương khái niệm `systemctl set-default` ở Câu 115 nhưng dành cho hệ init cũ.

🔑 **Từ khóa:** System V → `/etc/inittab` (`initdefault`); systemd → `systemctl set-default`.

Câu 120. Ý nghĩa symlink K/S trong thư mục rcN.d (System V init)

Question: Given the following two symbolic links in a System V init configuration:

`/etc/rc1.d/K01apache2` `/etc/rc2.d/S02apache2` When are the scripts executed that are referenced by these links? (Choose two.) (Dịch câu hỏi: Với 2 symlink `/etc/rc1.d/K01apache2` và `/etc/rc2.d/S02apache2`, các script này được chạy khi nào? (Chọn 2))

- **A.** `S02apache2` is run when runlevel 2 is entered.
- **B.** `S02apache2` is run when runlevel 2 is left.
- **C.** `K01apache2` is never run because K indicates a deactivated service.
- **D.** Both `S02apache2` and `K01apache2` are run during a system shutdown.
- **E.** `K01apache2` is run when runlevel 1 is entered.

Dịch nghĩa các đáp án:

- **A.** `S02apache2` chạy khi vào (entered) runlevel 2
- **B.** `S02apache2` chạy khi rời khỏi (left) runlevel 2 (sai — S chạy khi VÀO, không phải rời)
- **C.** `K01apache2` không bao giờ chạy vì K nghĩa là dịch vụ đã bị vô hiệu hóa (sai — K vẫn được chạy, chỉ với tham số "stop")
- **D.** Cả `S02apache2` và `K01apache2` đều chạy khi hệ thống shutdown (sai — `S02apache2` gắn với runlevel 2, không liên quan trực tiếp thao tác shutdown)
- **E.** `K01apache2` chạy khi vào (entered) runlevel 1

✅ **Answer: A, E**

Giải thích: Trong cơ chế System V init, mỗi thư mục `/etc/rcN.d/` chứa symlink trỏ tới script trong `/etc/init.d/`, đặt tên theo quy tắc:

- **S<số>tên** (Start) — script được chạy với tham số `start` khi **vào (entering)** runlevel đó. → `S02apache2` chạy khi vào runlevel 2 (A đúng).
- **K<số>tên** (Kill) — script được chạy với tham số `stop` khi **vào** runlevel chứa symlink K đó (không phải "rời" runlevel khác) — tức khi hệ thống chuyển **vào runlevel 1** (thường là single-user/maintenance mode), symlink `K01apache2` trong `/etc/rc1.d/` khiến Apache bị dừng lại (E đúng).

Số 2 chữ số sau S/K (01, 02) chỉ **thứ tự chạy** (chạy theo thứ tự tăng dần trong cùng runlevel), không mang ý nghĩa gì khác.

🔑 **Từ khóa:** `S` = Start (chạy khi VÀO runlevel đó); `K` = Kill (chạy khi VÀO runlevel đó, nhưng với tham số "stop"). Cả S và K đều gắn với hành động "vào" runlevel chứa chúng, không phải "rời".

TỔNG KẾT NHANH — CÁC ĐIỂM DỄ NHẦM CẦN

THUỘC LÒNG

Chủ đề`	Điểm mắ`u chố`t
<code>mkfs</code> không tham số`	→ ext2 (mặc định cổ điển)
<code>umask</code>	= base (666 file / 777 dir) trừ quyề`n mong muố`n
<code>tune2fs -i</code> vs <code>-c</code>	<code>-i</code> = interval (thời gian), <code>-c</code> = count (số` lầ`n mount)
<code>df</code> vs <code>du</code>	<code>df</code> = theo filesystem, <code>du</code> = theo file/thư mục
Hard link vs Symlink	Hard link: cùng filesystem, cùng inode, không trỏ được thư mục/khác fs. Symlink: trỏ đi đâu cũng được
<code>/usr/share/</code>	Dữ liệu dùng chung, không phụ thuộc kiế`n trúc CPU (man, docs...)
<code>nice</code> vs <code>renice</code>	<code>nice</code> = lúc khởi động mới; <code>renice</code> = đổi lúc đang chạy
Nice range	-20 (cao nhấ`t) → 19 (thấ`p nhấ`t); user thường chỉ tăng (0→19)
<code>nohup</code>	Miễn nhiệm SIGHUP khi logout
<code>source</code> / <code>.</code>	Chạy ngay trong shell hiện tại, không tạo subshell
Nháy đơn <code>'...'</code> vs nháy kép <code>"..."</code>	Đơn = literal; Kép = có mở rộng biế`n <code>\$VAR</code>
<code>tee</code>	Vừa in màn hình vừa ghi file
<code>which</code> vs <code>type</code>	<code>which</code> chỉ tìm theo <code>\$PATH</code> ; <code>type</code> biế`t cả builtin/alias
<code>></code> vs <code>>></code>	<code>></code> ghi đè; <code>>></code> nố`i thêm (append)
<code>2>&1</code> vs <code>1>&2</code>	Thứ tự số` quyế`t định chiề`u: <code>2>&1</code> (stderr→stdout), <code>1>&2</code> (stdout→stderr)
Ctrl+C / Ctrl+Z	C = SIGINT (ngắ`t); Z = SIGSTOP/SIGTSTP (tạm dừng)
Partition type ID	82=swap, 83=Linux, 8e=LVM, fd=RAID
MBR vs GPT	MBR: 4 primary, ~2.2TB; GPT: 128 partition, ~9.4ZB
<code>dpkg -r</code> vs <code>-P</code>	<code>-r</code> = remove (giữ config); <code>-P</code> = purge (xóa cả config)
<code>rpm -qa</code> vs <code>-ql</code>	<code>-qa</code> = list mọi gói; <code>-ql <pkg></code> = list file trong 1 gói
ext2/3/4 vs XFS/JFS	ext: inode cô` định từ lúc format; XFS/JFS: inode câ`p phát động
SetUID/SetGID/Sticky	4-2-1 (octal đầ`u): 4=SUID, 2=SGID, 1=Sticky
<code>/boot/</code> chứa gì	Chỉ kernel + initramfs + GRUB config (không chứa binary chương trình)
UEFI vs BIOS bootloader	UEFI → ESP (FAT32 riêng); BIOS → MBR (446 byte đầ`u ổ đĩa)
<code>isolate</code> vs <code>set-default</code>	<code>isolate</code> = đổi target ngay (tạm thời); <code>set-default</code> = đổi vĩnh viễn
System V <code>s</code> / <code>k</code> script	<code>s</code> = Start khi vào runlevel; <code>k</code> = Kill (stop) khi vào runlevel chứa nó
vi lặp lệnh	Số` đặt trướ c lệnh: <code>4j</code> , <code>2dd</code> , <code>3yy</code>

Chủ đề`	Điểm mắu chố t
vi lưu & thoát	<code>:wq</code> hoặc <code>ZZ</code> (hoa)
vi tìm kiế m	<code>/</code> = xuôi; <code>?</code> = ngược
<code>find -user</code> vs <code>-uid</code>	<code>-user</code> nhận tên; <code>-uid</code> nhận số`
<code>find -maxdepth</code>	Giới hạn độ sâu thư mục con
Container vs VM	Container = chung kernel host (nhẹ); VM = hypervisor + phầ n cứng ảo (mỗi VM 1 kernel riêng)

Chúc bạn ôn tập hiệu quả và thi đạt chứng chỉ LPIC-1 (101-500)! 🙌